Generative Psychometrics

The Ultimate Tutorial for AI-driven Scale Development in Generative Psychometrics: Releasing AIGENIE from its Bottle

Lara Russell-Lasalandra,[*†] Hudson Golino,[*†] Luis Garrido,[*‡] and Alexander Christensen[*¶]

†Department of Psychology, University of Virginia, Charlottesville, VA 22903, USA
‡Department of Psychology, Pontificia Universidad Madre y Maestra, Dominican Republic
¶Department of Psychology, Vanderbilt University, USA
*Corresponding author. Email: LLR7CB@Virginia.Edu; hfg9s@virginia.edu; luisgarrido@pucmm.edu.do; alexander.christensen@vanderbilt.edu

Abstract

Psychological scale development has traditionally required extensive expert involvement, iterative revision, and large-scale pilot testing before psychometric evaluation can begin. The `AIGENIE` R package implements the AI-GENIE framework (Automatic Item Generation with Network-Integrated Evaluation), which integrates large language model (LLM) text generation with network psychometric methods to automate the early stages of this process. The package generates candidate item pools using LLMs, transforms them into high-dimensional embeddings, and applies a multi-step reduction pipeline— Exploratory Graph Analysis (EGA), Unique Variable Analysis (UVA), and bootstrap EGA— to produce structurally validated item pools entirely *in silico*. This tutorial introduces the package across six parts: installation and setup, understanding Application Programming Interfaces (APIs), text generation, item generation, the `AIGENIE` function, and the `GENIE` function. Two running examples illustrate the package's use: the Big Five personality model (a well-established construct) and AI Anxiety (an emerging construct). The package supports multiple LLM providers (OpenAI, Anthropic, Groq, HuggingFace, and local models), offers a fully offline mode with no external API calls, and provides the `GENIE()` function for researchers who wish to apply the psychometric reduction pipeline to existing item pools regardless of their origin. The `AIGENIE` package is freely available on R-universe at https://laralee.r-universe.dev/AIGENIE.

Keywords: AIGENIE, Generative Psychometrics, tutorial, LLMs, AI, EGA, UVA, R package

1. Introduction

Large language models (LLMs) have become invaluable tools for scale development. Modern LLMs can generate extremely high-quality text Chakrabarty and Dhillon, 2026; Tengler and Brandhofer, 2025, and crucially, today's models function as powerful, expert-level writing tools straight out of the box. In other words, they are powerful enough to be used as-is without fine-tuning or retraining Carlini et al., 2021. Text generation, however, is only one piece of the puzzle. Encoder LLMs are also extremely powerful tools for psychometric applications Asudani et al., 2023; Tao et al., 2025, translating the context and meaning of human language into numeric, computer-readable vectors Vaswani et al., 2017. Given the considerable cost of traditional scale development (see Boateng et al. Boateng et al., 2018 & Clark and Watson Clark and Watson, 2016), researchers have begun

leveraging LLMs to streamline this process. A growing body of literature demonstrates that LLM-generated items can meet the same quality benchmarks expected of expert-authored items Götz et al., 2024; Hommel et al., 2022; Keane and McNaughton, 2026; Shin et al., 2025, and in some cases even surpass them Martin Kowal et al., 2025.

The present paper details how scale developers and methodologists can harness text generation models, embedding models, or *both* to accelerate scale development using an R package called `AIGENIE` (*Automatic Item Generation with Network-Integrated Evaluation* Russell-Lasalandra et al., 2024). `AIGENIE` is free for non-commercial purposes, completely open-source, and available on R-universe at https://laralee.r-universe.dev/AIGENIE.

This package serves an emerging research domain called **Generative Psychometrics** Garrido et al., 2025; Russell-Lasalandra et al., 2024; Russell-Lasalandra and Golino, 2026, in which language itself is treated as a resource that can be evaluated and assessed algorithmically. For example, Garrido et al. Garrido et al., 2025 used LLM-generated item pools and their embeddings to compare Principal Component Analysis (**PCA** Pearson, 1901) with network-based methods for recovering dimensional structure, demonstrating that psychometric questions can be investigated entirely through generated text without collecting human responses, and demonstrating the superiority of Exploratory Graph Analysis (**EGA** H. F. Golino and Epskamp, 2017) with item filtering Russell-Lasalandra et al., 2024 when compared to PCA. In `AIGENIE`, item content produced by LLMs (or by humans) is subjected to rigorous quantitative evaluation **prior to any human data collection**, enabling the development and structural validation of entire scales **in silico**. When the in silico structural organization of items is compared to the structural organization of variables obtained from nationally representative samples, the best-performing LLM models achieve a perfect match Russell-Lasalandra et al., 2024. That is, in silico structural validity can be equivalent to the structural validity recovered from human response data. This approach substantially reduces the resource barriers that have long characterized measurement development.

The `AIGENIE` methodology combines optional item generation with network psychometric techniques for structural validation. The package uses LLMs to generate large candidate item pools (or accepts an existing pool of human-authored items), embeds them as high-dimensional vectors via LLM embeddings, and then applies a multi-step psychometric pipeline to identify and remove redundant or unstable items. This pipeline includes Exploratory Graph Analysis (**EGA** H. F. Golino and Epskamp, 2017) for estimating dimensionality, Unique Variable Analysis (**UVA** Christensen et al., 2023) for detecting item redundancy, and bootstrap EGA (**bootEGA** Christensen and Golino, 2021) for evaluating the stability of items and dimensions within the EGA framework. The resulting item pool is a concise, structurally validated set ready for empirical testing. The efficacy of `AIGENIE` has been demonstrated through multiple large-scale Monte Carlo simulations across several LLMs and temperature settings, with results showing consistent improvements in structural validity across all conditions Russell-Lasalandra et al., 2024; Russell-Lasalandra and Golino, 2026.

The six steps of the AI-GENIE pipeline are as follows (see Figure 1):

- **Step 0: Generate or Write Your Initial Item Pool.** The item reduction process must begin with a sizable item pool. These initial items can either be generated seamlessly within the package using an LLM or directly supplied by the user.
- **Step 1: Embed items.** Each item is transformed into a high-dimensional numeric vector (an *embedding*) using an encoder LLM.
- **Step 2: Assess the initial item pool.** An EGA model is run on the initial item pool to estimate its dimensional structure before any reduction takes place. The detected communities (clusters of items) are compared to the known, intended structure using Normalized Mutual Information (**NMI** Danon et al., 2005). NMI can be thought of as an accuracy metric; in `AIGENIE`, NMI is displayed as a percentage value on a scale from 0–100%, where 0% indicates complete dissimilarity and 100% demonstrates perfect community detection. Higher NMI values indicate

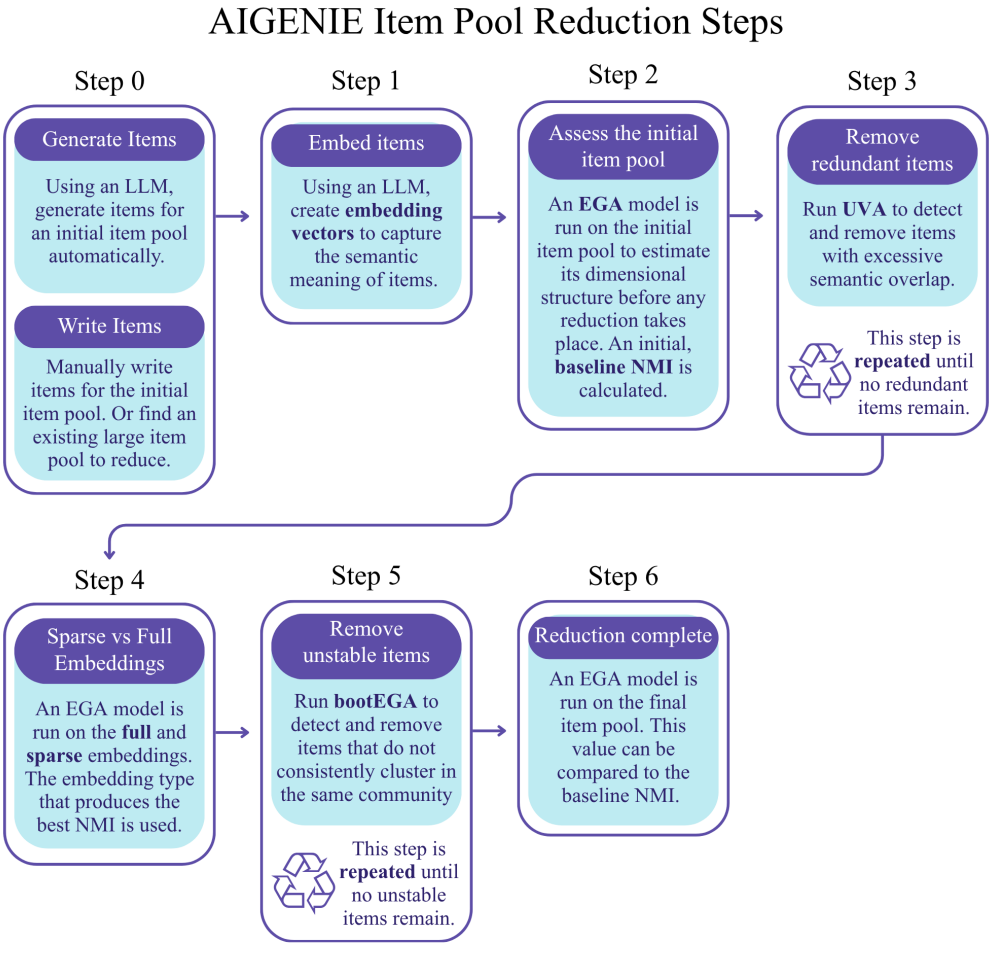

Figure 1. The six steps of the AIGENIE item pool reduction pipeline. Step 0 (which occurs before item reduction) is obtain the initial item pool (either by generating items using an LLM or manually writing them). Step 1 is generate the item embeddings using an LLM. Steps 2–6 involve using network psychometric techniques to whittle down the item pool algorithmically.

that the clusters within the EGA network better match the known item communities. This step establishes a baseline measure of structural validity.

- **Step 3: Remove redundant items.** UVA is used iteratively to detect and remove items with excessive semantic overlap. UVA identifies redundant item pairs or sets based on weighted topological overlap (**wTO** Zhang, Horvath, et al., 2005) within the network, retaining only the most unique representative from each redundant cluster. This step repeats until no further redundancies are found.
- **Step 4: Select sparse or full embeddings.** An EGA model is run on the *full* embedding matrix and a *sparsified* version of the matrix (in which only the most informative embedding dimensions are retained). The embedding type that corresponds to the EGA network with the best NMI is retained for all subsequent steps.
- **Step 5: Find the most stable items.** BootEGA is used to assess the structural stability of each item. In this step, 100 new embedding matrices are generated by drawing from a multivariate normal distribution parameterized by the original embedding matrix, and then EGA is applied to each one. If an item is consistently assigned to the same dimension across the 100 resamples, it is considered stable; items that frequently shift between communities are considered unstable and removed. This step repeats until all remaining items demonstrate high stability.
- **Step 6: Final pool is ready for review.** A final EGA model is run on the reduced item pool. A final NMI is calculated to assess the quality of the community detection after item pool reduction. This final NMI can be compared to that of the baseline.

The `AIGENIE` R package is an open-source toolkit that integrates artificial intelligence and network psychometrics. By automating labor-intensive stages of scale development such as item writing, redundancy pruning, and structural validation, the package reduces the time and financial burden of creating new measurement tools. Users can build a scale from scratch or refine an existing item pool, offering a flexible entry point into the emerging field of Generative Psychometrics. The tutorial that follows is intended to make every step of this process accessible, regardless of the reader's prior experience with LLMs or network psychometric analysis.

2. Tutorial Overview

The present tutorial provides a comprehensive, step-by-step guide to using the `AIGENIE` R package. This tutorial is organized into six parts that progressively introduce the package's capabilities:

- **Installation and Setup** covers the package installation from R-universe, Python environment configuration, and management of API-based (recommended for most researchers) and local LLMs.
- **Understanding APIs** (or *Application Programming Interface*) details the use of APIs to remotely access very powerful LLMs to generate items and embeddings.
- **Text Generation** introduces the text generation capabilities of the package through the `chat()` function, which allow users to interact with LLMs directly from R. Additionally, this section covers hyperparameter tuning and defining system roles.
- **Item Generation** demonstrates standalone item generation using the `AIGENIE()` function's `items.only` mode, which generates items without running the reduction pipeline. This section details the difference between the recommended "in-built" prompt and the ability to write custom prompts.
- **The `AIGENIE()` Function** walks through the complete AIGENIE pipeline from start to finish, first with a simple Big Five personality John and Srivastava, 1999 example and then with a novel, more extended AI Anxiety construct (**AIA** Wang and Wang, 2022) demonstration.
- **The `GENIE()` Function** provides users with detailed examples on how they can apply *only* the psychometric evaluation and reduction pipeline to user-supplied item pools. This capability is

for researchers who already have a large set of items that need to be checked for redundancy and structurally validated.

Throughout this tutorial, two running examples are used. The first is the well-established Big Five personality model John and Srivastava, 1999 to illustrate basic functionality, as many readers will have familiarity with this personality framework. The second is AI Anxiety (*AIA*) to illustrate the package's utility for developing scales for novel or underrepresented constructs. While AIA has received some growing research attention Güven et al., 2024; Li and Huang, 2020; X. Liu and Liu, 2025; Wang and Wang, 2022, it nonetheless lacks the robust literature presence of the "Big Five." In other words, this construct will likely be poorly represented in, or entirely absent from, the training data of many LLMs.

3. Installation and Setup

Setting up the `AIGENIE` package requires several steps: installing the package and its R dependencies, configuring the Python backend (which the package uses to communicate with LLMs), and installing support for fully local models (if your machine is powerful and has enough compute to run LLMs, which is not necessary to use the package). Although `AIGENIE` is an R package, it relies on Python in the background to communicate with LLMs; many of the software libraries used to call LLM APIs and run local models (e.g., the `llama-cpp-python` inference engine) are developed and maintained primarily in the Python. `AIGENIE` uses the `reticulate` R package Ushey et al., 2026 to call Python functions directly from R, giving users access to the full capabilities of these libraries *without ever needing to write or see Python code*. To be clear, `AIGENIE` offers a seamless, entirely R-native experience. It does, however, leverage the mature Python infrastructure that underlies much of the available modern LLM tooling.

Note that, for this section, there are platform-specific instructions for macOS/Linux and Windows.

3.1 Setup the Python Virtual Environment

`AIGENIE` communicates with LLMs through a Python backend. This Python environment is accessed seamlessly from R via the `reticulate` package Ushey et al., 2026. To manage the Python virtual environment, the package uses `uv` Astral, 2024, a fast, Rust-based utility that the `reticulate` ecosystem relies on to create virtual environments. Once the `uv` utility is successfully installed, close R (if running) and **fully restart your machine.**

3.1.1 MacOS/Linux Users

MacOS users need Apple's Command Line Tools to compile certain dependencies (e.g., `uv` utility).

If running macOS (*not* Linux), open the **Terminal** app and run the following command:

```
xcode-select --install
```

Note that Linux users should skip this particular command, as the necessary build tools are typically pre-installed or available through the system package manager.

If already installed, a message beginning with *Command line tools are already installed* will appear. Otherwise, a different dialog will appear prompting you to install the tools. Follow the on-screen instructions.

Next, install the `uv` utility on your system. In the **Terminal**, run the following command:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

If successful, you should see a message ending in *everything's installed!* It should **not** say *permission denied* anywhere in the output.

If you see a permission error, run the following commands in your **Terminal**:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc
```

Then, retry the `curl -LsSf https://astral.sh/uv/install.sh | sh` command.

3.1.2 Windows Users

If on a Windows computer, you will need to open the pre-installed **PowerShell** application to install `uv`. In a **PowerShell** window, run the following command:

```
irm https://astral.sh/uv/install.ps1 | iex
```

If successful, you should see a message ending in *everything's installed!* For further troubleshooting, see the uv installation documentation.

3.2 Install Package and Package Dependencies

Before downloading the `AIGENIE` package, all users should close R (if running) and **fully restart their computer** to ensure that R can locate the `uv` installation.

We recommend installing all package dependencies explicitly before installing `AIGENIE` itself. In an R script, run the following lines of code:

```
install.packages("reticulate")
install.packages("ggplot2")
install.packages("igraph")
install.packages("patchwork")
install.packages("tm")
install.packages("R.utils")
install.packages("jsonlite")
install.packages("EGAnet")
```

With the dependencies and `uv` utility in place, you can install the `AIGENIE` package. The package is available on R-universe, which provides pre-built binaries for all major operating systems. To grab the package from the R-universe, run the following:

```
# Install AIGENIE from R-universe
install.packages(
  "AIGENIE",
  repos = c("https://laralee.r-universe.dev",
  "https://cloud.r-project.org")
)
```

3.3 Confirm UV Installation

Once `AIGENIE` is installed, you can load the library and confirm the status of your `uv` installation:

```
library(AIGENIE)
check_installation <- python_env_info()
check_installation[["uv_available"]]  # This should be TRUE
```

If `uv_available` returns `FALSE` despite a successful installation, R may not be able to find `uv` on your system's PATH.

If running macOS, open the **Terminal** and run the following:

```
PATH="/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin:$HOME/.local/bin:
$HOME/.cargo/bin:$PATH"
```

If running Windows, open **PowerShell** and execute this command (*replace* `<you>` *with your computer Admin username*):

```
[Environment]::SetEnvironmentVariable("PATH", $env:PATH + ";C:\Users\
<you>\.local\bin", "User")
```

After applying either PATH fix, you must close your R script and **restart your machine** (not just restart the R session) for the changes to take effect.

3.3.1 Python Environment Setup

The first time you call any `AIGENIE` function that requires Python, the package will automatically create a dedicated Python virtual environment and install the necessary dependencies. This process typically takes 2–3 minutes on a standard internet connection and only needs to happen once. To setup the environment explicitly, run the `ensure_aigenie_python()` function. You can also use this function to customize the installation, if necessary. To reinstall the Python environment from scratch if you suspect something is awry, use the `reinstall_python_env()` function.

3.4 (Optionally) Install Local Model Support

If running LLMs on your own machine instead of relying on API service providers, you must install additional local model support. This support is required for the `local_AIGENIE()`, `local_GENIE()`, and `local_chat()` functions.

However, running models locally is neither necessary nor recommended for *most* users. Remotely accessing LLMs from providers like OpenAI and Anthropic will ensure access to frontier models that are *substantially* more powerful than the ones that can feasibly run on a personal computer. Models that run comfortably on typical machines (typically 7–13 billion parameters) are considerably less capable, and their generated items will generally be of lower quality. Running larger, more capable models locally (e.g., 70 billion parameters or above) demands significant computational resources (e.g., a high-end GPU). Local models are best suited for situations where data privacy requirements prohibit sending item content to external servers.

On macOS and Linux, local model support can be installed directly from within R by running the `install_local_llm_support()` function. For Apple Silicon Macs, this function automatically compiles `llama-cpp-python` with Metal acceleration for GPU-accelerated inference. For users with NVIDIA GPUs, additionally run `install_gpu_support()` to install CUDA-enabled PyTorch for accelerated embedding generation. Windows users need one prerequisite before this function will work: the Visual Studio C++ Build Tools from Microsoft.

With local support installed, you can download GGUF-format models from HuggingFace using the `get_local_llm()` function. For example, to download the Mistral 7B Instruct model capable of text generation, run the following lines of code

```
# The function returns the path where the model was saved
model_path <- get_local_llm(
  repo_id = "TheBloke/Mistral-7B-Instruct-v0.2-GGUF",
  filename = "mistral-7b-instruct-v0.2.Q4_K_M.gguf"
)
```

The message `model downloaded successfully` will appear in the console to indicate that the model has been downloaded to your machine. You can also verify the install by running `check_local_llm_se`

4. Understanding APIs

An Application Programming Interface (API) is a set of protocols that allow machines to communicate (see Auger et al. Auger and Saroyan, 2024). When `AIGENIE` generates items or embeddings using a cloud-based model, it does so by sending a request to the model provider's API. For example, `AIGENIE` could ask OpenAI's servers to run GPT-4o on a given prompt. Then, the API returns the model output to your computer. The user never interacts with the API directly; the package handles all communication implicitly behind the scenes.

To authenticate these requests, API providers require an **API key**: a unique string of characters that functions simultaneously as a password and an ID. When `AIGENIE` sends a request to, say, OpenAI's servers, it includes your API key so that the server can verify your identity, check that you have permission to use the requested model, and log the usage for billing purposes.

4.1 Best Practices for Using API Keys

There are several important practices to keep in mind when working with API keys.

1. Save your key immediately since providers will only display your key **once**. If you navigate away without copying it, you will need to generate a new one.
2. If you lose your key or suspect it may have been stolen, you can easily create a new key. Just remember to revoke (delete) your old key.
3. **Never share your key or include it in code that others can see** (e.g., public GitHub repositories). Anyone with your key can make requests that will be billed to your account.
4. Set a spending limit if possible. A spending cap protects against unexpected charges in the event that a key is compromised.
5. Be aware of rate limits. Each provider imposes limits on how frequently you can send requests within a given time window, which is tied to your API key. If you exceed these limits, the API will temporarily reject your requests.
6. Note that API usage is tracked in **tokens**, not words or characters. A token is a short word or word fragment. As a rule of thumb, one token is equivalent to roughly three-quarters of a word OpenAI, 2024b. So, one million tokens is equivalent to about 750,000 words. Providers typically charge separately for input tokens (the prompt you send) and output tokens (the text the model generates), with output tokens being priced higher.

4.2 Supported API Providers

The `AIGENIE` package supports API keys from five providers.

- OpenAI provides access to the GPT-family models (e.g., GPT-4o, GPT-5.1) for text generation. OpenAI also has excellent embedding models (recommended). The default embedding model in the package is their `text-embedding-3-small` model. A valid payment method is required to use an OpenAI API key, though costs for typical `AIGENIE` usage are *extremely* minimal (think cents, not dollars).
- Anthropic provides access to the Claude-family models for text generation. Anthropic requires that users prepay for API credits (minimum $5). By default, Anthropic will not overcharge you—once your prepaid credits are exhausted, requests will stop until you purchase more. However, $5 should be more than plenty for typical `AIGENIE` usage.
- Groq provides access to several open-source models (e.g., Llama, Mixtral, Gemma) for text generation. Groq created the Language Processing Unit (LPU Groq, n.d.), which is a hardware technology that enables extremely fast text generation. In fact, text generation is so efficient that low to moderate usage is completely **free**.

- HuggingFace is an open-source repository hosting thousands of LLMs for text generation and embedding. On HuggingFace, API Keys are called *tokens* (not to be confused with the *tokens* that represent small words or word fragments).
- Jina AI provides access to open-source embedding models. The free tier covers the first 10 million tokens.

4.3 Discovering Available Models

To see which models you have access to based on your API keys, use the `list_available_models()` function. For example, say you have valid OpenAI, Groq, and Anthropic API keys. The following code chunk could be used to determine which models your keys give you access to:

```
# List all available models across all providers
list_available_models(
  openai.API = "your-key", # ADD A VALID KEY!
  groq.API = "your-key", # ADD A VALID KEY!
  anthropic.API = "your-key" # ADD A VALID KEY!
)

# Filter by provider
list_available_models(provider = "groq", # shows only Groq models
                      groq.API = groq.API)

# Filter by type (text generation models vs. embedding models)
list_available_models(type = "chat", # text models available
                      openai.API = openai.API,
                      groq.API = groq.API,
                      anthropic.API = anthropic.API)

list_available_models(type = "embedding", # embedding models available
                      openai.API = openai.API,
                      groq.API = groq.API,
                      anthropic.API = anthropic.API)
```

5. Text Generation

Before diving into how to run the reduction pipeline on AI-generated (or human-written) items, we will first outline simple text generation. The `chat()` functions provide a straightforward interface for sending prompts to language models and receiving responses. These functions are useful for exploratory work or prompt testing. Throughout this section, the `chat()` function is used; however, the `local_chat()` could just as easily been used to demonstrate the same ideas with a locally installed model.

As a simple example, let's say we wanted to see what kind of items a an LLM would produce given minimal instructions to establish a sort of baseline. More specifically, we want the model generate items that target conscientiousness from the Big Five model of personality. A bare-bones command might be *"generate 5 items measuring conscientiousness for a personality scale."* The following code chunk and associated output shows GPT-4o's response to this prompt:

```
# Basic usage with OpenAI
response <- chat(
  prompts = "Generate 5 items measuring
  conscientiousness for a personality scale.",
```

```
  model = "gpt-4o",
  openai.API = "REDACTED" # the API key was removed
)

# See the response
response$response
```

```
[1] To measure conscientiousness on a personality scale, you
might consider items that assess traits such as organization,
dependability, diligence, and attention to detail. Here are
five example items:\n\n1. I plan my tasks carefully and
stick to my schedule to ensure everything gets done on time.
\n2. I pay close attention to details and make few mistakes
in my work.\n3. I am always prepared and rarely find myself
scrambling at the last minute.\n4. I follow through on my
commitments and can be relied upon to meet deadlines.\n5.
I keep my personal and work spaces organized and tidy.
\n\nEach item can be rated using a Likert scale, such as 1
(Strongly Disagree) to 5 (Strongly Agree), to assess the
level of conscientiousness in individuals.
```

5.1 Top p and Temperature

Two parameters that appear throughout the package— `temperature` and `top p`— influence how an LLM selects its next token during text generation. They directly affect the diversity and creativity of the items the model produces. In `AIGENIE`, both parameters default to 1.0, meaning no additional constraints are imposed beyond the model's own learned distribution. In this tutorial, we will demonstrate how output is affected by changing both the *temperature* and *top p* from their defaults. However, whenever a change is made to one of the two parameters, the other is left unchanged. In practice, it is common to adjust *one* parameter while leaving the other at its default rather than tuning *both* simultaneously.OpenAI, 2024a

5.1.1 Temperature

LLMs generate text one token at a time. At each step, the model assigns a probability to every possible next token in its vocabulary. A model's temperature value rescales this probability distribution before a token is sampled. Most models have a default temperature of 1.0; at this temperature, the model samples directly from its learned probabilities. Lowering the temperature sharpens the distribution, making high-probability tokens even more likely to be chosen. Doing so produces more predictable, repetitive, and less complex output. Raising the temperature flattens the distribution, giving lower-probability tokens a better chance of being selected and producing more varied (but potentially less *coherent*) output.

The effect of temperature on output quality (if there even is an effect Patel et al., 2024) is not straight forward and depends on the context as well as the specific model Renze, 2024; Zhu et al., 2024. Temperature can be though of as a "creativity knob," where tasks that require higher creativity may benefit from higher temperatures (though, even this relationship to creativity is not always a given Peeperkorn et al., 2024). For item generation within `AIGENIE`, simulation results Russell-Lasalandra and Golino, 2026 showed that AI-GENIE reliably improved structural validity across all temperature settings (0.5, 1.0, and 1.5), though the relationship between temperature and item quality was not always straightforward and depended on the model used.

In the example above where GPT-4o was asked to *generate 5 items measuring conscientiousness for a personality scale*, the `temperature` was left at the default of 1. However, we can modify the optional

`temperature` argument to see how temperature tuning influences the output. Let's increase the temperature to 1.5 and observe its effects:

```
# Generate a response to the question at a higher temperature
response_high_temp <- chat(
  prompts = "Generate 5 items measuring conscientiousness
  for a personality scale.",
  model = "gpt-4o",
  openai.API = "REDACTED", # API key removed
  temperature = 1.5 # Increased to a HIGHER temp
)

# View the results
response_high_temp$response
```

```
[1] Sure, here are five items designed to measure
conscientiousness for a personality scale:\n\n1.
**Dutifulness**  \n   - I consistently meet my obligations
even if they feel inconvenient or challenging.\n\n2.
**Orderliness**  \n   - I prefer to have a well-
organized environment and consistently maintain order
in my daily tasks and workspace.\n\n3. **Dependability**
\n   - People can count on me to follow through on
commitments without needing reminders.\n\n4. **Attention
to Detail**  \n   - I pay close attention to detail and
take care to ensure accuracy in everything I do.\n\n5.
**Self-Discipline**  \n   - I ensure to finish tasks I
start, even if they are dull, rather than allowing
distractions to undermine my productivity. \n\nThese
items assess various aspects of conscientiousness, such
as dutifulness, orderliness, dependability,
meticulousness, and self-discipline.
```

In general, the items in the output produced by the higher temperature GPT-4o model are more lexically complex and verbose than the items in the output using the default model. For example, the item *I keep my personal and work spaces organized and tidy* came from the default temperature model, whereas the item *I prefer to have a well-organized environment and consistently maintain order in my daily tasks and workspace* came from the higher-temperature model.

We can also decrease the `temperature` argument to a value below 1 to investigate how a lower temperature setting may impact the output:

```
# Generate a response to the question at a lower temperature
response_low_temp <- chat(
  prompts = "Generate 5 items measuring conscientiousness
  for a personality scale.",
  model = "gpt-4o",
  openai.API = "REDACTED", # API key removed
  temperature = 0.5 # Decreased to a LOWER temp
)

# View the results
response_low_temp$response
```

```
[1] Certainly! Here are five items designed to measure
conscientiousness for a personality scale:\n\n1. **I a
m meticulous and pay attention to details in my work
and personal tasks.**\n   - Strongly Disagree\n   -
Disagree\n   - Neutral\n   - Agree\n   - Strongly Agree
\n\n2. **I often plan my activities in advance and follow
a structured schedule.**\n   - Strongly Disagree\n   -
Disagree\n   - Neutral\n   - Agree\n   - Strongly Agree
\n\n3. **I am reliable and can be counted on to fulfill
my commitments and responsibilities.**\n   - Strongly
Disagree\n   - Disagree\n   - Neutral\n   - Agree\n   -
Strongly Agree\n\n4. **I prefer to keep my living and
working spaces organized and tidy.**\n   - Strongly
Disagree\n   - Disagree\n   - Neutral\n   - Agree\n
- Strongly Agree\n\n5. **I am persistent and work
diligently to achieve my goals, even when faced with
obstacles.**\n   - Strongly Disagree\n   - Disagree\n
- Neutral\n   - Agree\n   - Strongly Agree\n\nThese
items are designed to assess various aspects of
conscientiousness, such as attention to detail,
organization, reliability, planning, and persistence.
```

In this particular example, the items from the lower temperature model are fairly similar to that of the default temperature model. In fact, the item *I prefer to keep my living and working spaces organized and tidy* from the lower-temperature model is almost identical to the item *I keep my personal and work spaces organized and tidy* from the default temperature model.

However, the effects of temperature tuning may not be obvious across a single instance; increasing the `reps` argument (which controls the number of times the model responds to the provided prompt) from the default of 1 to 10 or 50 might reveal some systematic differences that are not immediately obvious in a single output. Although, model hyperparameter tuning is a far less important than model selection Evstafev, 2025. That is, determining whether GPT-4o is the best LLM for the task is paramount; then, determining which temperature setting of GPT-4o should be used can be considered.

5.1.2 Top p

Top p (also called nucleus sampling) simply truncates the next-token probability distribution rather than rescale it Holtzman et al., 2019. The model considers only the smallest set of tokens whose cumulative probability exceeds the threshold *p*, and samples from that subset. For example, a `top p` of 0.9 means the model considers only the most probable tokens that together account for 90% of the probability mass. The remaining 10% of unlikely tokens are completely discarded. A `top p` less than the default of 1 restricts the model to a narrower set of high-probability tokens. Thus, the default setting of `AIGENIE` includes the LLM's entire token vocabulary and effectively disables nucleus sampling.

Consider the same example from above: we want the GPT-4o model to generate items that target conscientiousness given a bare-bones prompt (i.e., *generate 5 items measuring conscientiousness for a personality scale*). Let's decrease the `top p` parameter to 0.5:

```
# Generate a response to the question at a lower top p
response_low_top_p <- chat(
  prompts = "Generate 5 items measuring conscientiousness
  for a personality scale.",
  model = "gpt-4o",
```

```
  openai.API = "REDACTED", # API key removed
  top.p = 0.5 # Decreased to a LOWER top p value
)

# View the results
response_low_top_p$response
```

```
[1] Certainly! Here are five items designed to measure
conscientiousness for a personality scale:\n\n1. **Task
Completion**: I always complete my tasks thoroughly and on
time, even when they are challenging or tedious.\n\n2.
**Organization**: I keep my workspace and personal areas
organized and tidy, ensuring everything is in its proper
place.\n\n3. **Attention to Detail**: I pay close attention
to details and strive for accuracy in everything I do,
avoiding careless mistakes.\n\n4. **Dependability**: Others
can rely on me to fulfill my commitments and responsibilities
consistently.\n\n5. **Goal-Oriented**: I set clear goals for
myself and work diligently to achieve them, even when it
requires sustained effort over time.\n\nThese items aim to
capture various aspects of conscientiousness, such as
reliability, organization, diligence, and attention to detail.
```

These items, when compared to those generated with the default `top p` setting, exhibit a noticeably more uniform sentence structure. Each item begins with an independent clause and ends with a clarifying subordinate phrase (e.g., *I set clear goals for myself and work diligently to achieve them, even when it requires sustained effort over time*). By contrast, when `top p` was left at the default value of 1.0, the syntactic structure of the generated items varied more naturally. For instance, the item *I keep my personal and work spaces organized and tidy* contains a single verb phrase, whereas *I plan my tasks carefully and stick to my schedule to ensure everything gets done on time* chains multiple verb phrases together.

5.2 Defining a System Role

Until this point in the tutorial, we've only experimented with changes to the `user prompt`. The user prompt, or the prompt that most people are readily familiar with, is the actual request or instruction. However, another type of input is a **system role** prompt. A system role is a set of background instructions that shape how the model behaves Chen et al., 2025. It functions like an identity or persona that the LLM should embody as it generates the output.

For example, a system role might be something like *you are an expert psychometrician and test developer specializing in personality assessment*. This instruction tells the model to approach item generation from the perspective of a domain expert. This primes the model toward domain-consistent language, professional tone that a generic prompt alone might not elicit A. Liu et al., 2024. Research on the efficacy of persona prompting has shown that assigning a role to a model can improve the quality and coherence of its output, particularly for tasks that benefit from domain-specific framing De Paoli, 2023; Jiang et al., 2024; Kong et al., 2024. However, in some contexts, the benefits tend to only increase when the assigned role is well-aligned with the task domain Zheng et al., 2024. And, sometimes, adding a system role may have unforeseen limitations Hu et al., 2026. However, in the context of item generation, persona prompting will very likely only have a positive or negligible effect on output.

To add a system role, do the following:

```
# Define a helpful model persona
```

```
system.role <- "You are an expert psychometrician who
specializes in personality measurement. You know how to
write clear, concise items that are robust."

# Generate a response to the question with a system role
response_system_role <- chat(
  prompts = "Generate 5 items measuring conscientiousness
  for a personality scale.",
  model = "gpt-4o",
  openai.API = "REDACTED", # API key removed
  system.role = system.role # System role prompt provided
)

# View the results
response_system_role$response
```

```
[1] Certainly! Here are five items designed to measure
conscientiousness for a personality scale. Please rate
each item on a scale from 1 (Strongly Disagree) to 5
(Strongly Agree):\n\n1. I am meticulous in my approach to
organizing and completing tasks.\n2. I plan my activities
in advance to ensure that I meet deadlines consistently.
\n3. I pay close attention to details in both my personal
and professional life.\n4. I often set goals for myself
and work diligently to achieve them.\n5. I hold myself
accountable for completing tasks to the best of my ability.
```

With a system role, delivers the items directly, using more precise, psychometrically conventional language (e.g., *I am meticulous in my approach to organizing and completing tasks* rather than *I pay close attention to details and make few mistakes in my work*). The system role shifts the model's posture from educator explaining the task to *practitioner executing the task*, producing output that is closer to the register expected of a finished assessment instrument.

6. Item Generation

One of the core capabilities of the `AIGENIE` package is automated item generation. Given a set of constructs and their attributes, the package prompts an LLM to generate novel candidate items. This section demonstrates how to generate items without running the full psychometric pipeline, using the `items.only = TRUE` flag within either the `AIGENIE()` or its local equivalent `local_AIGENIE()` functions. Generating items in isolation is useful for exploring what the model produces and refining prompts iteratively.

6.1 The item.attributes Parameter

The single most important parameter in the `AIGENIE()` function is `item.attributes`. This parameter is a named list in which each element represents an *item type* (i.e., a construct or dimension of interest), and the character vector within each element specifies the *attributes*. Attributes are the specific facets, themes, or content areas that the generated items should collectively cover.

Consider the five traits within the "Big Five" personality model John and Srivastava, 1999 (openness to experience, consciousness, extraversion, agreeableness, and neuroticism). Each personality trait encompasses several behaviors; for example, someone who is exhibits high levels of openness to experience may be very artistic, but that person could just as easily enjoy philosophical discussion.

Being "creative" and "philosophical," therefore, describe two valid manifestations of the same personality trait. Building on this idea of defining trait manifestations for each of the five traits, we can create the following `item.attributes` object:

```
big5_attributes <- list(
  openness = c("creative", "perceptual", "curious",
  "philosophical"),
  conscientiousness = c("organized", "responsible", "disciplined",
  "prudent"),
  extraversion = c("friendly", "positive", "assertive",
  "energetic"),
  agreeableness = c("cooperative", "compassionate", "trustworthy",
  "humble"),
  neuroticism = c("anxious", "depressed", "insecure",
  "emotional")
)
```

Here, the five names of the list (openness, conscientiousness, etc.) are the item types, and the character vectors within each are the attributes. The model will be instructed to generate items that target each of these attributes, producing items across the full breadth of each construct.

It should be noted that attributes do *not* need to correspond to formal subscales or established subdomains. They simply represent the range of content that the items, as a whole, should cover. If a researcher is developing a unidimensional scales should not think of attributes as separate subscales; they reflect the distinct thematic facets that a comprehensive set of a given type of item should span. Including attributes ensures that the LLM does not generate items that cluster around a single narrow aspect of the construct while neglecting others. In `AIGENIE`, the model is instructed to produce items for each attribute, which results in a more content-diverse initial pool.

Additionally, while the attributes within this tutorial are one word, attributes can just as well be richer, more verbose phrases if prudent. The the number of attributes per item type through this tutorial are consistent (e.g., there are four attributes per OCEAN personality trait). However, the number of attributes per item type *can* vary (e.g., *openness* could have eight attributes whereas *neuroticism* could have only three), so long as there are at least two for any given item type.

6.2 Generating Items within the AIGENIE Function

Generating items using the `AIGENIE()` or `local_AIGENIE()` function offers two distinct modes for how prompts are constructed and sent to the model. The choice between them determines how much control the user has over the exact wording of the instructions the LLM receives.

6.2.1 The In-Built Prompt

In the **built-in prompting mode** (the default), the user provides as many descriptive components as possible and the package automatically assembles them into a complete, well-structured prompt behind the scenes (See Figure 2). These descriptive components are analogous bricks; the `AIGENIE()` function arranges, aggregates, and assembles these "bricks" into a stable, coherent "structure." This "structure" is the primary `user prompt` passed to the LLM. Using this in-built prompt is recommended for most researchers because it incorporates the prompt engineering strategies shown to produce high-quality items in simulation studies Russell-Lasalandra et al., 2024, and it avoids the risk of omitting critical prompt components that can degrade output quality.

The descriptive components (or "bricks") that the user can provide in the built-in prompting mode are all **optional**, but users should provide **as many as possible**. These components are as follows:

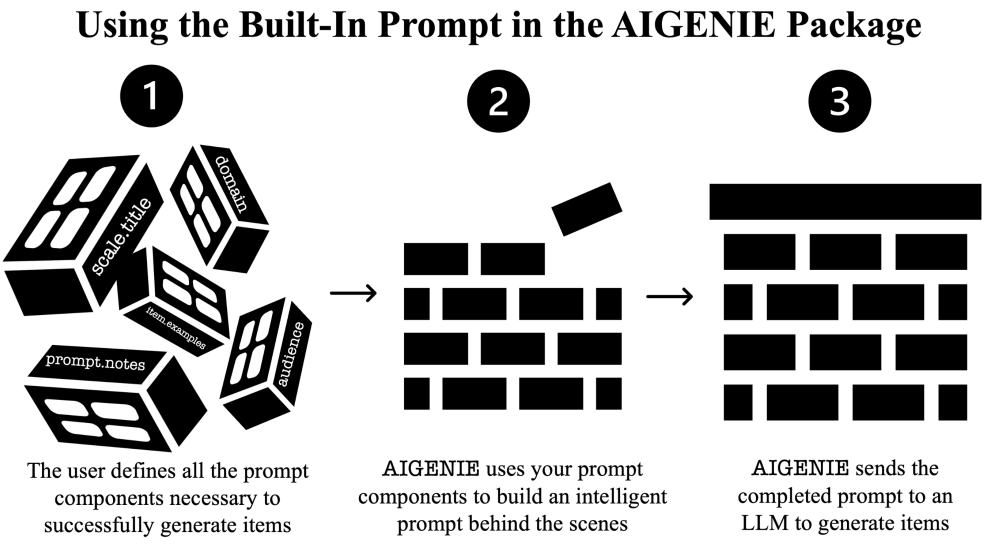

Figure 2. When using AIGENIE in the in-built prompt mode, the function takes the user-specified components (e.g., domain, prompt.notes, item.type.definitions, or scale.title) and uses them to build a strong prompt automatically.

- `domain` specifies the research domain (e.g., `"personality measurement"` or `"child development"`).
- `scale.title` provides the names of the scale being developed.
- `audience` describes the scale's intended target population (e.g., `"college-educated adults"` or `"children with ASD in second grade"`).
- `item.type.definitions` is a named list providing a brief definition of each item type, giving the model substantive context about the construct.
- `response.options` specifies the intended response options for the items (e.g., `c("disagree", "neutral"`
- `item.examples` is a data frame containing high-quality example items to guide the model's generation style and format.
- `prompt.notes` allows the user to append custom instructions to the automatically constructed prompt without having to write the entire prompt from scratch (e.g., *"ensure every item begins with the stem 'I am someone who'..."* or *"items should be very brief and contain no words that would exceed a fifth-grade vocabulary"*). That is, the user can inject specific requirements or constraints while still benefiting from the package's built-in prompt engineering. If the built-in prompt handles most of what you need and only a small adjustment is required, `prompt.notes` is the right tool for the job.

Previously, we defined the `big5_attributes` object to demonstrate the `item.attributes` parameter. Now that the parameters pertaining to the in-built prompt have been defined, we can write code to generate items using the `AIGENIE()` function:

```
# First.. define the important item.attributes object
big5_attributes <- list(
  openness = c("creative", "perceptual", "curious",
  "philosophical"),
  conscientiousness = c("organized", "responsible", "disciplined",
  "prudent"),
  extraversion = c("friendly", "positive", "assertive",
  "energetic"),
```

```
  agreeableness = c("cooperative", "compassionate", "trustworthy",
  "humble"),
  neuroticism = c("anxious", "depressed", "insecure",
  "emotional")
)

# Generate items using the built-in prompt
items_builtin <- AIGENIE(
  item.attributes = big5_attributes, # defined above
  openai.API = "REMOVED", # API Key REMOVED
  model = "gpt-4o",

  # Descriptive components for prompt construction
  domain = "personality measurement",
  scale.title = "Big Five Personality Inventory",
  audience = "college-educated adults in the United States",
  item.type.definitions = list(
    openness = "Openness reflects intellectual curiosity,
        aesthetic sensitivity, and a preference for novelty
        and variety.",
    conscientiousness = "Conscientiousness reflects a tendency
        toward self-discipline, goal-directed behavior, and
        organization.",
    extraversion = "Extraversion reflects sociability,
        assertiveness, and the tendency to seek stimulation
        in the company of others.",
    agreeableness = "Agreeableness reflects a tendency to be
        cooperative, compassionate, and trusting toward
        others.",
    neuroticism = "Neuroticism reflects emotional instability,
        including proneness to anxiety, sadness, and mood
        swings."
  ),
  response.options = c("strongly disagree", "disagree",
    "neutral", "agree", "strongly agree"),
  prompt.notes = "All items should be written as first-person
    self-report statements beginning with 'I am someone who'.",

  # System role... the model's persona
  system.role = "You are an expert psychometrician and test
    developer specializing in personality assessment.",

  target.N = 8, # Generating only 8 items per item type
  items.only = TRUE # ONLY generating items in this example
)

# View the some of the resulting items
items_builtin$statement[1:5]
```

```
[1] I am someone who often finds unconventional solutions to
```

```
problems.
[2] I am someone who enjoys engaging in activities that allow
me to express my imaginative ideas.
[3] I am someone who notices details in the environment that
others might overlook.
[4] I am someone who is able to detect subtle differences in
tone and mood during conversations.
[5] I am someone who seeks out new knowledge and experiences
for the sake of learning.
```

6.2.2 Implicit Prompt Engineering within AIGENIE

The quality of AI-generated items depends heavily on how the LLM is prompted. As explained above, when no custom prompts are provided, `AIGENIE()` automatically constructs prompts using the information supplied through its parameters. Recent simulation work within the AI-GENIE framework has demonstrated that prompt engineering strategies can substantially influence the structural validity and redundancy of generated item pools, with effects that scale with model capability Russell-Lasalandra and Golino, 2026. Therefore, this section makes a note of which prompt engineering strategies are used implicitly when using the in-built prompt.

The default prompt architecture incorporates several prompt engineering best practices:

- System role (persona prompting): A domain-specific expert persona is assigned to the model, built from the `domain`, `scale.title`, and `audience` parameters.
- Contextual instructions: The prompt includes the construct definition (from `item.type.definitions`), the target attributes, and the response format.
- Few-shot examples: If `item.examples` are provided, they are incorporated to anchor the model's output style.
- Adaptive generation: when `adaptive = TRUE` (the default), previously generated items are appended to the prompt to prevent repetition. A follow-up simulation study Russell-Lasalandra and Golino, 2026 further demonstrated that the prompt engineering strategy of **adaptive prompting** Lightman et al., 2023 can meaningfully shape the quality of AI-generated item pools, with effects that scale with model capability. Adaptive prompting is a prompting strategy where the LLM is shown a running list of everything it has already produced and explicitly instructed not to repeat or rephrase any of those earlier items. Typically, LLMs left unconstrained tend to regurgitate the same ideas over and over again. Adaptive prompting counteracts that tendency by making the model's output from previous iterations part of its input context.

6.2.3 The Custom Prompt

In the **custom prompting mode**, the user supplies fully written prompts via the `main.prompts` parameter. There must be exactly one prompt per item type. In this mode, the user has complete control over the exact instructions the model receives. This mode is best suited for researchers who want to incorporate specific prompt engineering strategies or exercise fine-grained control over every aspect of how the model is instructed.

However, custom prompting is substantially more demanding than the built-in mode and is **not** generally recommended for *most* users. When the package constructs prompts automatically, it includes several components that are easy to overlook when writing prompts from scratch. For instance, prompt-writers must consider clear task framing and communication, the explicit mention of all the item type's attributes, decisions on the number of items to generate per model instance, and diligent outlining of all important context the model needs. Omitting any one of these components can lead to items that are poorly formatted or off-target at best, and items that are unparseable or otherwise unusable for reduction analysis at worst. Thus, **we recommend that most researchers**

use the built-in mode and only switch to custom prompting after gaining familiarity with the package and with prompt engineering more broadly.

For researchers who do choose custom prompting, there are several requirements and best practices to follow. First, `main.prompts` must be a **named list with one prompt per item type**, and the names must match the names in `item.attributes` exactly. Second, each prompt **must explicitly reference all of a given item type's attributes** listed in the corresponding element of `item.attributes`. The package uses these attributes to parse and label the returned items; if an attribute is missing from the prompt, items targeting that facet will not be generated or will it be labeled correctly. Third, each prompt should be self-contained. The prompt should provide all the context the model needs for that particular item type, because **each prompt is sent to the model independently**.

A well-constructed custom prompt *typically* includes the following components, in roughly this order:

- **Task context**: a good description of what is being generated and why.
- **Contextual background**: details like a strong definition of the target construct, who the scale is intended for, and all other pertinent information.
- **Explicit generation instructions**: the model must know how many items to generate and how they should be distributed across attributes (this component is **mandatory**).
- a list of the attributes, named exactly as they appear in `item.attributes` (this component is **mandatory**).
- **Quality constraints**: these constraints can include instructions to generate novel items, avoid existing measures, and use a specific item format.

Let's examine an example of a custom prompt. Recall that the `big5_attributes` object defined in previous examples listed attributes for each of the Big Five personality traits (e.g., "curiosity" and "philosophical" were named for `openness` where as "friendly" and "positive" were named for `extraversion`).

The `big5_attributes` object was used to generate items using the in-built prompt, and we will use it again to guide our prompt construction for this custom prompting example. To achieve the same result using custom prompts, the we must write a complete, self-contained prompt for each of the five item types. Every prompt must reference all attributes by name exactly as they appear in `big5_attributes`. Here's what these prompts might look like:

```
# Create the "main.prompts" object
custom_prompts <- list(
  openness = "You are generating novel items targeting the Big Five
    personality trait openness to experience. Openness to experience
    is a personality trait that describes how open-minded, creative,
    and imaginative a person is. Generate EXACTLY eight items total
    for openness to experience; generate two items per attribute of
    openness to experience. These attributes are as follows: 1)
    creative, 2) perceptual, 3) curious, and 4) philosophical. Do
    NOT add or remove any attributes; use the attributes EXACTLY as
    provided. All items should be first-person self-report
    statements beginning with 'I am someone who'. Do NOT look for
    items that already exist in the literature; all items should
    be novel. Don't be afraid to push the bounds of the construct.",

  conscientiousness = "You are generating novel items targeting the
    Big Five personality trait conscientiousness. Conscientiousness
```

```
    is a personality trait that describes one's tendency toward self-
    discipline, goal-directed behavior, and organization. Generate
    EXACTLY eight items total for conscientiousness; generate two
    items per attribute of conscientiousness. These attributes are as
    follows: 1) organized, 2) responsible, 3) disciplined, and 4)
    prudent. Do NOT add or remove any attributes; use the attributes
    EXACTLY as provided. All items should be first-person self-report
    statements beginning with 'I am someone who'. Do NOT look
    for items that already exist in the literature; all items should
    be novel. Don't be afraid to push the bounds of the construct.",

  extraversion = "You are generating novel items targeting the Big
    Five personality trait extraversion. Extraversion is a personality
    trait that describes people who are more focused on the external
    world than their internal experience. Generate EXACTLY eight
    items total for extraversion; generate two items per attribute of
    extraversion. These attributes are as follows: 1) friendly, 2)
    positive, 3) assertive, and 4) energetic. Do NOT add or remove
    any attributes; use the attributes EXACTLY as provided. All items
    should be first-person self-report statements beginning with 'I
    am someone who'. Do NOT look for items that already exist in the
    literature; all items should be novel. Don't be afraid to push
    the bounds of the construct.",

  agreeableness = "You are generating novel items targeting the Big
    Five personality trait agreeableness. Agreeableness is
    personality trait that describes one's tendency to be
    cooperative, compassionate, and trusting toward others. Generate
    EXACTLY eight items total for agreeableness; generate two items
    per attribute of agreeableness. These attributes are as follows:
    1) cooperative, 2) compassionate, 3) trustworthy, and 4) humble.
    Do NOT add or remove any attributes; use the attributes EXACTLY
    as provided. All items should be first-person self-report
    statements beginning with 'I am someone who'. Do NOT look for
    items that already exist in the literature; all items should be
    novel. Don't be afraid to push the bounds of the construct.",

  neuroticism = "You are generating novel items targeting the Big Five
    personality trait neuroticism. Neuroticism is a personality trait
    that describes one's tendency to experience negative emotions
    like anxiety, depression, irritability, anger, and self-
    consciousness. Generate EXACTLY eight items total for
    neuroticism; generate two items per attribute of neuroticism.
    These attributes are as follows: 1) anxious, 2) depressed, 3)
    insecure, and 4) emotional. Do NOT add or remove any attributes;
    use the attributes EXACTLY as provided. All items should be first-
    person self-report statements beginning with 'I am someone who'.
    Do NOT look for items that already exist in the literature; all
    items should be novel. Don't be afraid to push the bounds of the
```

```
    construct."
)
```

Notice that each prompt contains task context, construct definitions, attribute lists, quantity instructions, and quality constraints. Each prompt follows the same general template but must be individually tailored to the target construct (i.e., item type), and the attributes must be listed by name exactly as they appear in `big5_traits`. If even one attribute is misspelled or omitted, the package will not be able to parse and label the resulting items correctly. Moreover, if the researcher later decides to change the item stem from "I am someone who" to, say, "I tend to," that change must be applied manually across all five prompts in the custom mode. In the built-in mode it requires editing only the single `prompt.notes` string.

With these prompts drafted, we can call the `AIGENIE()` function to generate items. Note that parameters like `audience` and `response.options` are not supplied here as they were when using the in-built prompt (the "bricks" are useless if you already have a sturdy "wall/structure" made):

```
# Generate items using the custom prompts
items_custom <- AIGENIE(
  item.attributes = big5_attributes, # defined previously
  openai.API = "REMOVED", # API key removed
  model = "gpt-4o",

  main.prompts = custom_prompts, # these are the custom prompts

  # Give the model the same persona as before
  system.role = "You are an expert psychometrician and test developer
    specializing in personality assessment.",
  target.N = 8, # only generate 8 items per item type
  items.only = TRUE # only return items... no reduction pipeline
)

# View the first 5 items generated
items_custom$statement[1:5]
```

```
[1] I am someone who finds novel ways to express my thoughts
and ideas.
[2] I am someone who enjoys transforming ordinary objects into
something artistic.
[3] I am someone who notices subtle details in my surroundings
that others might miss.
[4] I am someone who enjoys immersing myself in new sensory
experiences, like tasting unfamiliar foods or listening to
diverse music genres.
[5] I am someone who feels a strong need to investigate and
understand how things work.
```

7. The AIGENIE Function

The full AI-GENIE pipeline automates the entire workflow from item generation through structural validation. A single call to the `AIGENIE()` function (or local equivalent `local_AIGENIE()` function) generates items, embeds them, estimates the network structure, removes redundant items, assesses stability, and returns a validated item pool. This section walks through the complete pipeline using two examples: a simple Big Five demonstration and a more extended AI Anxiety application.

7.1 Understanding the Output of the AIGENIE Function

This section describes the **default output** that *most* researchers will see when they work with `AIGENIE` (i.e., the object returned when `items.only`, `embeddings.only`, `run.overall`, `keep.org`, and `all.together` are all set to their default value of `FALSE`). Variations introduced by each flag are described at the end of this section.

To better help demonstrate the structure of the output, let's begin with a focused example generating items for three of the Big Five traits. Note that this code is identical to the example listed under text generation, but with the `items.only` flag removed, the target number of items increased to a more sizable count appropriate for reduction, and the model changed to a more capable one:

```
# Generate items and run the reduction pipeline
reduction_builtin <- AIGENIE(
  item.attributes = big5_attributes, # defined previously
  openai.API = "REMOVED", # API Key REMOVED
  model = "gpt-5.1", # using a more modern model

  # Descriptive components for prompt construction
  domain = "personality measurement",
  scale.title = "Big Five Personality Inventory",
  audience = "college-educated adults in the United States",
  item.type.definitions = list(
    openness = "Openness reflects intellectual curiosity,
        aesthetic sensitivity, and a preference for novelty
        and variety.",
    conscientiousness = "Conscientiousness reflects a tendency
        toward self-discipline, goal-directed behavior, and
        organization.",
    extraversion = "Extraversion reflects sociability,
        assertiveness, and the tendency to seek stimulation
        in the company of others.",
    agreeableness = "Agreeableness reflects a tendency to be
        cooperative, compassionate, and trusting toward
        others.",
    neuroticism = "Neuroticism reflects emotional instability,
        including proneness to anxiety, sadness, and mood
        swings."
  ),
  response.options = c("strongly disagree", "disagree",
    "neutral", "agree", "strongly agree"),
  prompt.notes = "All items should be written as first-person
    self-report statements beginning with 'I am someone who'.",

  # System role... the model's persona
  system.role = "You are an expert psychometrician and test
    developer specializing in personality assessment.",

  target.N = 60, # Generating 60 items per item type
)
```

7.1.1 Top-Level Structure

The default output is a named list with two top-level elements: `item_type_level` and `overall`:

```
# See the top-level structure of the output
names(reduction_builtin)
```

```
[1] item_type_level   overall
```

The `item_type_level` object is itself a named list containing one element per item type, named to match `item.attributes`. In our Big Five example, `reduction_builtin$item_type_level` contains five sublists corresponding to each of the five personality traits: `openness`, `conscientiousness`, `extraversion`, `agreeableness`, and `neuroticism`. Each sublist holds the complete set of results from the reduction pipeline as applied to that item type **in isolation**.

The `overall` object is also a named list that aggregates the final items and embeddings across all item types into a single object. The final items remaining after the reduction analysis for items of all types are stored as a data frame in the `overall` object: `reduction_builtin$overall$final_items`. The `overall` object also contains an element called `embeddings`. The `embeddings` object is a list containing the *sparsified* and the *full* embedding matrices used in the reduction pipeline.

7.1.2 Results on the Item Type Level

Since items are evaluated in the reduction pipeline completely agnostic of items that are of other types, separate, independent results are available for **each** of the item types. In other words, contentiousness items go through the reduction pipeline **only** with other conscientiousness times, neuroticism items go through the reduction pipeline **only** with other neuroticism, and so on. Each per-type sublist within `reduction_builtin$item_type_level` contains thirteen elements. Therefore, in our example, there would be thirteen result elements pertaining to **each** of the five personality traits. These are the thirteen elements:

```
# Pick a trait (openness) to see what results are available on the
# item-type level in general
names(reduction_builtin$item_type_level$openness)
```

```
 [1] final_NMI          initial_NMI        embeddings
 [4] UVA                bootEGA            EGA.model_selected
 [7] final_items        final_EGA          initial_EGA
[10] start_N            final_N            network_plot
[13] stability_plot
```

Here is closer look at each of these thirteen elements:

- `final_items` is the most practically important element. It is a `data.frame` containing the items that survived the full reduction pipeline. Its columns are:
  - `ID`: A numeric identifier assigned during generation.
  - `statement`: The item's actual text (e.g., *I often lose myself in creative projects*)
  - `attribute`: The attribute the item was generated to reflect (e.g., "creative").
  - `type`: The item type the item belongs to (e.g., "openness").
  - `EGA_com`: The community assignment from the final EGA network.
- `start_N` records the total number of items generated for this item type. In our example, `reduction_builtin$item_type_level$agreeableness$start_N` would return 64. Thus, there were 64 `agreeableness` items in the initial item pool before reduction.

- `final_N` records the total number of items generated for this item type. In our example, `reduction_builtin$item_type_level$agreeableness$final_N` would return 49. Thus, there were 49 `agreeableness` items in the final item pool after the reduction analysis.
- `initial_NMI` is a numeric value reporting the Normalized Mutual Information (NMI) between the network-detected community structure and the known attribute assignments **before** any reduction steps.
- `final_NMI` is a numeric value reporting the NMI between the network-detected community structure and the known attribute assignments **after** all reduction steps.
- `UVA` is a list containing results from the Unique Variable Analysis (UVA) redundancy reduction step. UVA identifies pairs or sets of items whose embeddings are so similar that retaining them all would introduce redundancy. The list contains:
  - `n_removed` is the total number of redundant items removed across all UVA sweeps.
  - `n_sweeps` is the number of iterative passes UVA required before no further redundancies were detected.
  - `redundant_pairs` is a `data.frame` logging every redundancy decision. Each row records the sweep in which the redundancy was detected, the items involved, which item was kept, and which item or items were removed.
- `bootEGA` is a list containing results from the bootstrap Exploratory Graph Analysis (bootEGA) step. After UVA removes semantically redundant items, bootEGA evaluates the structural stability of the remaining items by repeatedly resampling the embedding matrix and estimating the network structure. Items that frequently change community membership across bootstrap samples are pruned. The list contains:
  - `initial_boot` is the `bootEGA` object (from the `EGAnet` package H. Golino and Christensen, 2025) estimated on the post-UVA item pool, before any stability-based pruning.
  - `final_boot` is the `bootEGA` object estimated after stability-based pruning.
  - `n_removed` reports the number of items removed due to low structural stability across all bootEGA stability sweeps.
  - `items_removed` is a `data.frame` logging the specific items that were identified and pruned during the stability check.
  - `initial_boot_with_redundancies` is a `bootEGA` object estimated on the full, pre-UVA item pool. This object is useful for comparing the stability of the item pool before and after redundancy reduction.
- `EGA.model_selected` is a character string indicating which network estimation model was selected: Triangulated Maximally Filtered Graph (**TMFG** Massara et al., 2016) or Extended Bayesian Criterion Graphical Least Absolute Shrinkage and Selection Operator (**EBICglasso** or **glasso** Foygel and Drton, 2010; Friedman et al., 2008). When `EGA.model = NULL` (the default), the package tests both models and selects whichever produces a higher NMI. If the user specifies a model via the `EGA.model` parameter, this field simply reflects that choice.
- `initial_EGA` is an EGA object (from the `EGAnet` package) representing the network structure of the item pool **before** the reduction pipeline.
- `final_EGA` is an EGA object representing the network structure **after** the reduction pipeline
- `embeddings` is a list containing the embedding matrices used in the analysis. The list contains three elements:
  - `full` is the dense (full) embedding matrix for the final retained items. Rows are embedding dimensions, columns are items (column names correspond to item IDs).
  - `sparse` is the sparsified embedding matrix for the final retained items. Sparsification zeroes out values in the middle 95% of the distribution, retaining only the most extreme embedding entities.

  - `selected` names the embedding type (either full or sparse) used in the analysis. The package tests both and selects whichever yields a higher NMI.

- `network_plot` stores the `ggplot`Wickham, 2016/`patchwork`Pedersen, 2025 object displaying a side-by-side comparison of the EGA networks before and after reduction with the initial network on the left and the final network on the right. NMI values are annotated on each panel. An example of this network plot for `agreeableness` items is shown in Figure 3.
- `stability_plot` is a `ggplot/patchwork` object comparing item stability before (the plot on the left) and after (the plot on the right) reduction. Item stability refers to how consistently each item is assigned to the same community across bootstrap samples. Higher stability values (closer to 1) indicate that an item reliably belongs to its assigned dimension. An example of such a plot for `agreeableness` items is shown in Figure 4.

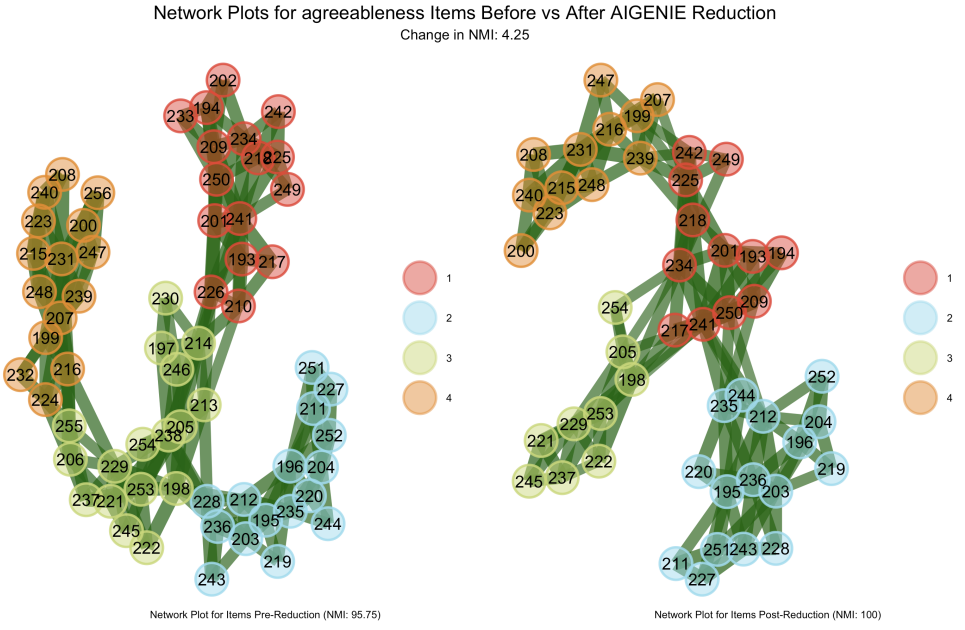

Figure 3. A plot showing the EGA network before item reduction (on the left) compared to the network after item reduction (on the right) for Agreeableness items.

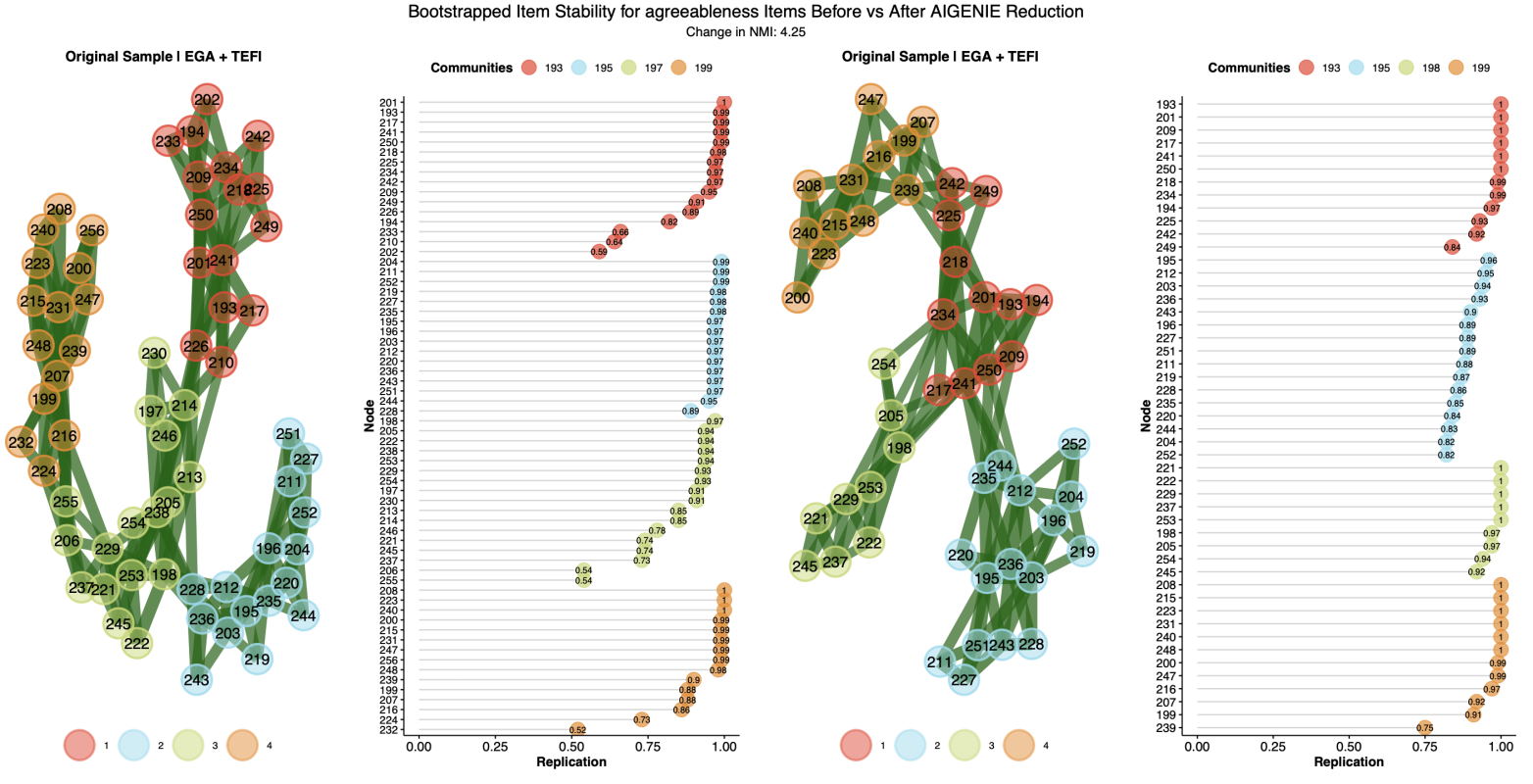

Figure 4. The stability of the EGA network before reduction (left) versus after (right) reduction. The stability plot shows a massive improvement in stability post-reduction. Additionally, the NMI has increased by about 4%

7.1.3 Output Variations by Flag

The aforementioned output structure occurs under default settings and is likely sufficient for many researchers. However, this output structure may change, depending on if any of these four flags are changed from the default of `FALSE` to `TRUE`: `items.only`, `embeddings.only`, `keep.org`, and `run.overall`. Additionally, the output changes when the flag `all.together = TRUE`, but in a more fundamental way; this flag is covered in Section 7.1.4.

As discussed previously, when `items.only = TRUE`, the function skips embedding, network analysis, and reduction entirely. It returns a simple `data.frame` with four columns (`ID`, `statement`, `type`, and `attribute`). This data frame contains the raw generated item pool. This is useful when the researcher wants to inspect or manually curate items before running the psychometric pipeline, or when items will be embedded externally and passed to `GENIE()`.

When `embeddings.only = TRUE`, the function generates items and computes embeddings but skips the psychometric reduction pipeline. It returns a named list with two elements: `embeddings` (the embedding matrix) and `items` (the items `data.frame` described above).

When `keep.org = TRUE`, the top-level structure remains the same (`item_type_level` and `overall`), but each per-type sublist gains an `initial_items` field. This field contains a data frame of all items generated *before* reduction. The `embeddings` sublists also gain `full_org` and `sparse_org` matrices corresponding to the full pre-reduction item pool. The `overall` element similarly gains `initial_items`, `full_org`, and `sparse_org`.

When `run.overall = TRUE`, the `item_type_level` results are unchanged, but the `overall` element becomes a full analysis object rather than a simple aggregation. It includes its own `final_NMI`, `initial_NMI`, `EGA.model_selected`, `final_EGA`, `initial_EGA`, `start_N`, `final_N`, and `network_plot`. Critically, **this overall analysis does *not* perform additional reduction—** it evaluates the combined post-reduction item pool as a whole, which is useful for assessing cross-trait dimensionality.

7.1.4 The all.together Flag

By default, the reduction pipeline processes each item type independently. In our example, for instance, openness items are embedded, analyzed, and pruned without any knowledge of the conscientiousness items, and vice versa. This strategy is likely the most appropriate strategy for researchers.

Setting `all.together = TRUE` changes this behavior ***fundamentally***. Instead of running separate pipelines for each item type, the function pools every generated item into a single batch and runs the full reduction pipeline on the entire pool simultaneously. This means that UVA can now detect and remove redundancies that span item types (e.g., a "warmth" item under agreeableness that is semantically indistinguishable from a "friendliness" item under extraversion), and EGA estimates a single network structure across all items at once.

Internally, when `all.together = TRUE`, the package reassigns each item's `attribute` to a concatenation of its original type and attribute (e.g., "openness creative", "neuroticism anxious") and collapses all items into a single nominal type. The pipeline then proceeds as though there were only one item type, and NMI is computed against this concatenated attribute structure.

The output structure also changes. Rather than the two-level `item_type_level` / `overall` organization, the function returns a flat named list containing: `final_NMI`, `initial_NMI`, `embeddings`, `UVA`, `bootEGA`, `EGA.model_selected`, `final_items`, `final_EGA`, `initial_EGA`, `start_N`, `final_N`, `network_plot`, and `stability_plot`. These are the same elements described in the per-type breakdown above, but applied to the combined pool.

When the boundaries between item types are theoretically fuzzy and the researcher suspects that constructs may share substantial semantic overlap, altering this flag may be a good exploratory step. Additionally, when the researcher wants to let the network structure emerge entirely from the data rather than imposing a type-level separation a priori, changing this flag may prove fruitful.

Note that `all.together` is ignored when only one item type is present, since there is no distinction between per-type and pooled reduction in that case.

Lastly, it's is worth explicitly contrasting `all.together` with `run.overall` since their names are so similar. The `run.overall` flag does *not* change how reduction works— items are still pruned independently within each type. It simply adds a post-hoc EGA analysis of the combined final pool to assess cross-trait structure. Whereas with `all.together`, the reduction itself operates on the combined pool, meaning that items can be removed because of cross-type redundancy. The two flags answer different questions. The `run.overall` flag asks *what does the cross-trait structure look like after per-type reduction?* While the `all.together` flag asks *what happens when we let the reduction pipeline see all items at once?*

7.2 Running AIGENIE on An Emerging Construct: AI Anxiety

Now that the `AIGENIE()` function use and output has been demonstrated using the established "Big Five" personality model, we now turn to a more realistic application: developing a scale to measure anxiety about AI. AI Anxiety (**AIA** Wang and Wang, 2022) is a construct of growing research interest that currently lacks a well-established, comprehensive measurement instrument. Thus, AIA is an ideal candidate for demonstrating the package's value for developing scales where well established instruments likely do not exist within the LLM training data.

Wang & Wang Wang and Wang, 2022 identified a four-factor structure based on distinct sources of anxiety that individuals experience in response to the development, deployment, and societal integration of AI technologies. These four factors are:

- **Learning anxiety** is cognitive overwhelm in the face of AI complexity, including perceiving one's knowledge as insufficient for AI demands and feeling daunted by the pace of AI advancement.
- **Job replacement** describes fear of professional obsolescence, including anxiety that AI will eliminate one's professional role, belief that one's skills can be automated, and uncertainty about career stability.
- **Sociotechnical blindness** is concern about societal AI impacts, including loss of autonomy to AI systems, concerns about AI-enabled privacy violations, and worry about over-reliance on AI technology.
- **AI configuration** describes unease regarding AI system opacity, including lacking confidence in AI decision-making, confusion about how AI systems operate, and doubting AI's reliability and safety.

The original AI-GENIE paper included a small-scale simulation using this construct Russell-Lasalandra et al., 2024, and we follow the same operationalization here. When creating the `item.attributes` object for AIA, it is important to consider the central themes of each factor:

```
# Defining the item.attributes object for AIA
ai_anxiety_attributes <- list(
  learning_anxiety = c("overwhelmed", "inadequacy", "intimidated"),
  job_replacement = c("threatened", "replaceable", "insecure"),
  sociotechnical_blindness = c("powerless", "overly dependent",
  "surveilled"),
  ai_configuration = c("distrustful", "uncertain",  "vulnerable")
)
```

With the `item.attributes` object defined, only a few more elements are needed before a scale can be generated:

```
# Provide detailed definitions for each factor
ai_anxiety_definitions <- list(
  learning_anxiety = paste(
    "Learning anxiety refers to cognitive overwhelm in the face of AI complexity.",
    "It includes perceiving one's knowledge as insufficient for AI demands",
    "and feeling daunted by the pace of AI advancement."
  ),
  job_replacement = paste(
    "Job replacement anxiety refers to fear of professional obsolescence.",
    "It includes fearing that AI will eliminate one's professional role,",
    "believing one's skills can be automated, and experiencing uncertainty",
    "about career stability."
  ),
  sociotechnical_blindness = paste(
    "Sociotechnical blindness refers to concern about societal AI impacts.",
    "It includes loss of autonomy to AI systems, concerns about AI-enabled",
    "privacy violations, and worrying about over-reliance on AI technology."
  ),
  ai_configuration = paste(
    "AI configuration anxiety refers to anxiety about AI system opacity.",
    "It includes lacking confidence in AI decision-making, confusion about",
    "how AI systems operate, and doubting AI's reliability and safety."
  )
)

# Run the full pipeline
ai_anxiety_results <- AIGENIE(
  item.attributes = ai_anxiety_attributes,
  openai.API = "REMOVED", # API Key removed
  model = "gpt-5.1",
  embedding.model = "text-embedding-3-small",
  domain = "technology-related psychological assessment",
  scale.title = "AI Anxiety Scale",
  audience = "adults who use or are exposed to AI technologies in daily life",
  item.type.definitions = ai_anxiety_definitions,
  response.options = c("strongly disagree", "disagree", "slightly disagree",
                       "slightly agree", "agree", "strongly agree"),
  target.N = 80
)
```

The results from this singular run were promising. The NMI consistently trended upward post-reduction (see Figures 5, 6, 7, and 8).

8. The GENIE Function

Not all researchers need AI-generated items, hence, the advent of `GENIE`. This function performs all the same reduction steps as `AIGENIE` without the LLM-generation step (that is, `AIGENE` without "AI"). The `GENIE()` function (or its local equivalent `local_GENIE`) applies the full network psychometric evaluation pipeline (embedding, EGA, UVA, bootEGA) to any user-supplied set of items, without generating any new content.

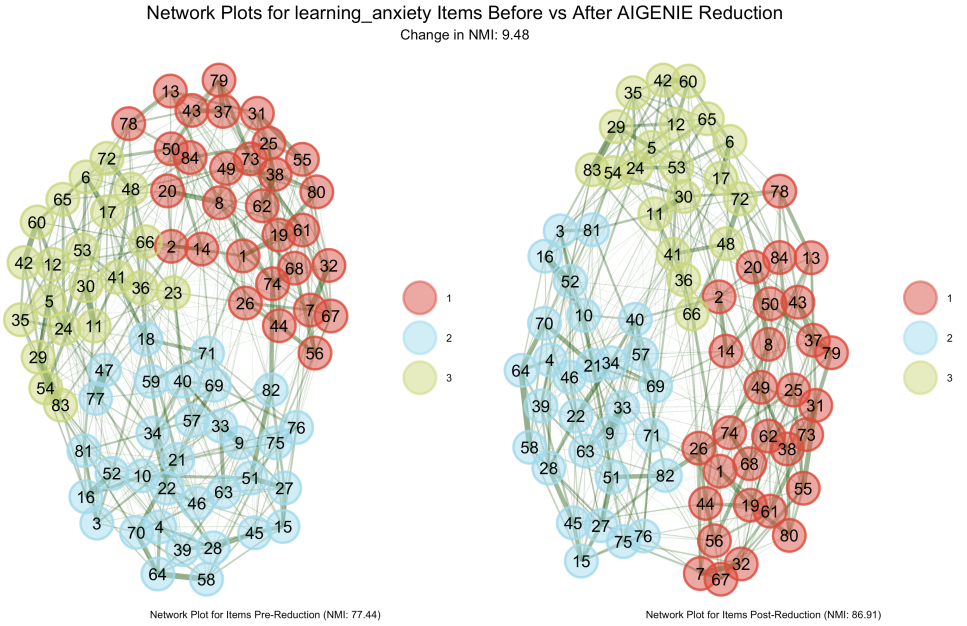

Figure 5. The EGA network before (left) vs after (right) reduction for AI Learning anxiety items. The NMI improved by 9.48%. The final NMI is 86.91%

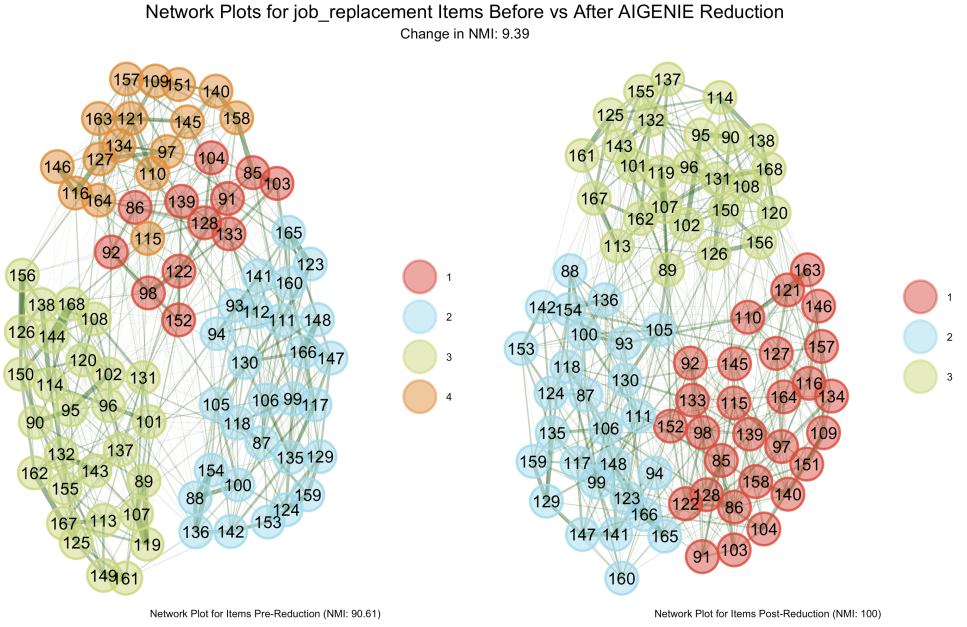

Figure 6. The EGA network before (left) vs after (right) reduction for AI Job Replacement items. The NMI improved by 9.39%. The final NMI is 100%.

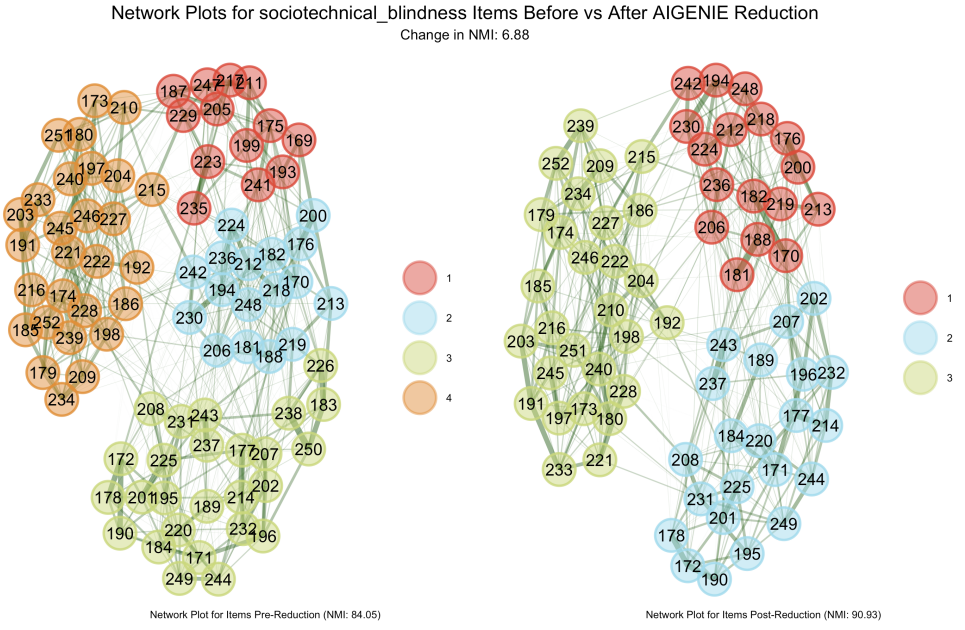

Figure 7. The EGA network before (left) vs after (right) reduction for AI Configuration items. The NMI improved by 6.88%. The final NMI is 90.93%.

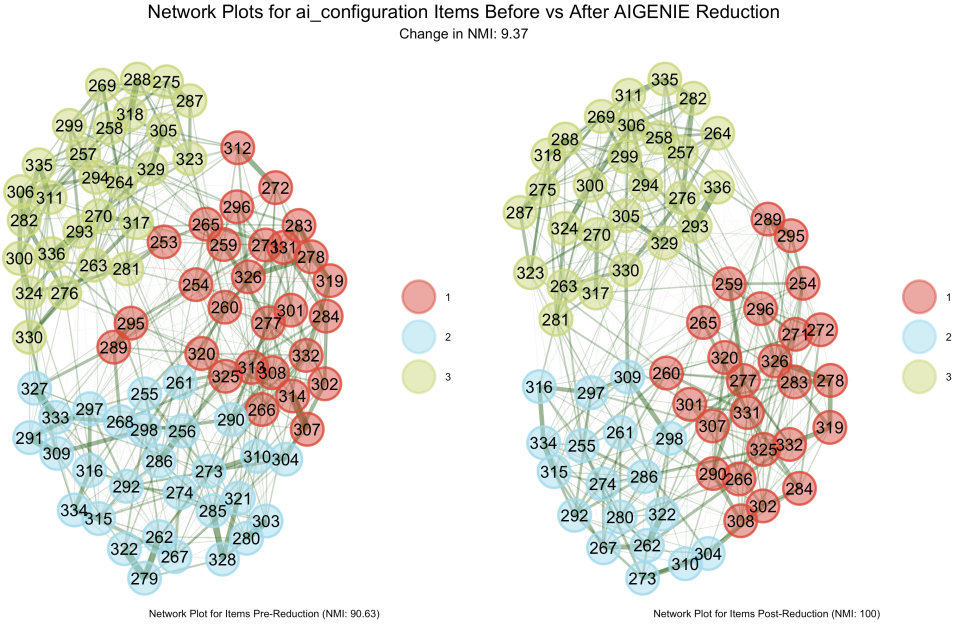

Figure 8. The EGA network before (left) vs after (right) reduction for AI Configuration items. The NMI improved by 9.37%. The final NMI is 100%.

8.1 Using GENIE with AIA Items

The `GENIE()` function requires a data frame with four columns: `statement` (the item text), `attribute` (the sub-facet or characteristic the item targets), `type` (the overarching construct), and `ID` (a unique identifier). The structure of this `data.frame` is identical to the data frame that is returned in `AIGENIE` when the `items.only` flag is set to `TRUE`.

To better explicate the use of the `GENIE` function, we will continue considering the emerging construct AIA discussed in Section 7.2. Firstly, an initial item pool needs to be specified and loaded into the environment. We have provided a **miniature** initial item pool. **This item pool contains too few items (only 5 per item type) for a meaningful reduction analysis**, but is nonetheless still useful to understand the expected structure of the required data frame:

```
# Example structure showcasing how GENIE expects the items to be
# formatted
my_ai_anxiety_items <- data.frame(
  statement = c(
    # Learning anxiety items
    "I feel overwhelmed by how quickly AI technology is advancing",
    "I worry that my knowledge is not enough to keep up with AI",
    "The complexity of AI systems makes me feel inadequate",
    "I am intimidated by the amount I would need to learn about AI",
    "I feel daunted when I try to understand how AI works",
    # Job replacement items
    "I worry that AI will make my job obsolete",
    "The thought of AI replacing human workers makes me anxious",
    "I am concerned that AI will take over my career field",
    "I feel insecure about my professional future because of AI",
    "I fear that my skills will become irrelevant as AI improves",
    # Sociotechnical blindness items
    "I feel powerless in the face of AI-driven decisions that affect me",
    "I worry about becoming too dependent on AI technology",
    "It bothers me that AI can track and predict my behavior",
    "I worry about losing my privacy to AI-powered surveillance",
    "I am concerned about how much autonomy I am giving up to AI",
    # AI configuration items
    "I do not trust AI systems to make important decisions",
    "I am uncertain about how AI systems actually arrive at their answers",
    "It concerns me that I cannot verify whether AI output is reliable",
    "I feel vulnerable when I have to rely on AI I do not understand",
    "I doubt that AI systems are safe enough to be widely deployed"
  ),
  attribute = c(
    rep("overwhelmed", 2), rep("inadequacy", 2), "intimidated",
    rep("threatened", 2), rep("replaceable", 2), "insecure",
    "powerless", "overly dependent", rep("surveilled", 2), "powerless",
    "distrustful", rep("uncertain", 2), "vulnerable", "distrustful"
  ),
  type = c(
    rep("learning_anxiety", 5),
    rep("job_replacement", 5),
    rep("sociotechnical_blindness", 5),
```

```
    rep("ai_configuration", 5)
  ),
  ID = paste0("AI_", 1:20),
  stringsAsFactors = FALSE
)
```

With items prepared, running the `GENIE` function is straightforward (a much larger dataset must be used, so this code only serves as a reference and was not actually run):

```
# Run the full GENIE pipeline on your items
genie_results <- GENIE(
  items = my_ai_anxiety_items, # ENSURE LARGE ENOUGH
  openai.API = "REDACTED" # API Key REMOVED
)
```

The output of the `GENIE()` function is the same as the output of the `AIGENIE()` function, so reviewing the results of the `GENIE` output can be done easily, especially if one already has some familiarity with the `AIGENIE()` function.

8.2 Using GENIE with Existing Embeddings

If you have already embedded your items (perhaps using a different tool or during a previous session), you can supply the embedding matrix directly to skip the embedding step:

```
# Suppose you have a pre-computed embedding matrix
# (rows = embedding dimensions, columns = items, colnames = item IDs)
# my_embeddings <- [your embedding matrix here]

genie_results_precomputed <- GENIE(
  items = my_ai_anxiety_items,
  embedding.matrix = my_embeddings  # Provide your own embeddings
)
```

Skipping the embedding step is particularly useful for researchers who want to compare how different embedding models affect the structural analysis, or who want to embed items using a specialized model not natively supported by the package. However, a vast majority of users will very likely want to use one of the native methods. Additionally, if the embeddings are provided to the `GENIE()` function, no API calls (nor their keys) are required. Therefore, using `GENIE()` to replicate and share results may be ideal.

9. Discussion

The present tutorial has introduced the `AIGENIE` R package, a comprehensive tool for automated psychological scale development and structural validation. The package implements the AI-GENIE framework Russell-Lasalandra et al., 2024, integrating LLM-based item generation with network psychometric methods to produce structurally validated item pools efficiently. This R package allows users an easy and practical way to dive into the emerging realm of **Generative Psychometrics**.

We have demonstrated the package's core capabilities from basic installation and setup, through text generation and embedding, to the full psychometric pipeline. Two running examples (the well-established Big Five personality model John and Srivastava, 1999 and the emerging construct of AI Anxiety Wang and Wang, 2022) illustrated the package's utility across varying levels of construct maturity in the literature.

The most immediate advantage of working within this framework is the substantial reduction in the time and cost required to produce a structurally valid initial item pool that has been checked for redundancy. Traditional scale development typically requires a team of content experts to draft items, multiple rounds of review and revision, and large-scale pilot testing before psychometric evaluation can even begin — a process that can span months or years and cost tens of thousands of dollars Fenn et al., 2020. `AIGENIE` compresses much of this early-stage work into a single function call that generates, embeds, and psychometrically evaluates an item pool in minutes. By packaging state-of-the-art prompt engineering, text embedding, and network psychometric methods into an accessible R interface, `AIGENIE` lowers the barrier to entry for scale construction. This could lead to a broader and more diverse set of psychological assessments being available, particularly for constructs that are culturally specific, newly emerging, or otherwise underserved by existing instruments.

Additionally, the deterministic and source-agnostic nature of the reduction pipeline has implications beyond AI-generated items. Any researcher with an existing item pool— whether expert-authored, adapted from prior instruments, or compiled from qualitative research— can submit it to the same embedding, UVA, and bootEGA pipeline for an objective structural evaluation. Thus, the `AIGENIE` approach surpasses any method that uses the output of LLMs to *evaluate* item quality (e.g., asking an LLM whether an item is robust) in terms of its replicability. Additionally, `AIGENIE` avoids the issue of using an LLM to evaluate an LLM, which raises a whole host of potential red flags.

9.1 Limitations and Considerations

Several limitations should be kept in mind when using this tool. First, while AI-GENIE provides structural validation in silico, it does not replace the need for empirical validation with human participants. The generated and reduced item pools should be considered strong candidates for empirical testing, not finished instruments. The original AI-GENIE paper demonstrated that in silico structural validity closely tracks empirical structural validity for the best-performing models Russell-Lasalandra et al., 2024, but this correspondence should not be taken for granted in every application.

However, while `AIGENIE` **does *not* eliminate the need for expert oversight, it *does* fundamentally change where human judgment is most needed.** `AIGENIE` shifts the focus away from the laborious drafting of initial items and toward the higher-order tasks of reviewing, refining, and empirically validating a pre-curated pool.

Additionally, the quality of generated items depends **heavily** on the LLM used. Newer and more capable models generally produce better results, particularly when combined with advanced prompting strategies Russell-Lasalandra and Golino, 2026. However, the LLM landscape is evolving rapidly— models that are state-of-the-art at the time of writing will most definitively be superseded by the time this tutorial is read. The package's provider-agnostic architecture is designed to accommodate this evolution.

Finally, the `AIGENIE` package is under active development. The latest version of `AIGENIE` can always be found on R-universe at https://laralee.r-universe.dev/AIGENIE, and the full source code is publicly available for inspection and contribution. We welcome feedback, feature requests, and contributions from the research community.

Appendix 1. Quick Reference

Appendix 1.1 Core Functions

Table 1 provides a complete list of the functions available in the `AIGENIE` package.

Appendix 1.2 Combining Providers

The `AIGENIE` package allows mixing providers for text generation and embedding. For example, a researcher could use Groq for fast item generation with an open-source model while relying on OpenAI for embeddings:

Table 1. Functions available in the `AIGENIE` package.

Function	Description
`AIGENIE()`	Full pipeline: generate items, embed, EGA, UVA, bootEGA
`GENIE()`	Validation pipeline for user-provided items
`chat()`	Send prompts to supported LLMs
`list_available_models()`	List models available across providers
`local_AIGENIE()`	Run full pipeline with locally hosted LLMs
`local_GENIE()`	Validate items with local models
`local_chat()`	Chat with local models
`ensure_aigenie_python()`	Configure Python environment
`python_env_info()`	Show environment details
`reinstall_python_env()`	Rebuild Python environment
`set_huggingface_token()`	Configure Hugging Face access
`install_local_llm_support()`	Install local LLM dependencies
`install_gpu_support()`	Enable GPU acceleration
`check_local_llm_setup()`	Verify local LLM configuration
`get_local_llm()`	Download local LLM models

```
results <- AIGENIE(
  item.attributes = item_attributes,
  groq.API = "your-groq-key",
  openai.API = "your-openai-key",
  model = "llama-3.3-70b-versatile",
  embedding.model = "text-embedding-3-small",
  target.N = 60
)
```

Alternatively, Anthropic's Claude models can be paired with Jina AI embeddings:

```
results <- AIGENIE(
  item.attributes = item_attributes,
  anthropic.API = "your-anthropic-key",
  jina.API = "your-jina-key",
  model = "sonnet",
  embedding.model = "jina-embeddings-v3",
  target.N = 60
)
```

Appendix 1.3 Querying Available Models

Model availability changes as providers update their catalogs. The current list of available models can be queried directly:

```
# Per-provider queries
list_available_models("openai",    openai.API = openai_key)
list_available_models("groq",      groq.API = groq_key)
list_available_models("anthropic", anthropic.API = anthropic_key)
list_available_models("jina")
```

```
# All providers at once
list_available_models(
  openai.API    = openai_key,
  groq.API      = groq_key,
  anthropic.API = anthropic_key
)

# Filter by type
list_available_models(openai.API = openai_key, type = "chat")
list_available_models(openai.API = openai_key, type = "embedding")
```

Appendix 1.4 Supported Chat Models

Table 2 lists commonly used chat models and their shorthand aliases supported by `AIGENIE`. Note that this is a reference snapshot; the `list_available_models()` function always returns the current catalog.

Table 2. Commonly used chat models and their shorthand aliases.

Provider	Models	Aliases
OpenAI	gpt-4o, gpt-4-turbo, gpt-5.1, gpt-5.2	gpt4o, chatgpt
Anthropic	claude-opus-4, claude-opus-4.6	sonnet, opus, haiku, claude
Groq	llama-3.3-70b-versatile, qwen-2.5-72b	llama3, mixtral, gemma, qwen

Appendix 1.5 Supported Embedding Models

Table 3 lists commonly used embedding models.

Table 3. Commonly used embedding models.

Provider	Models
OpenAI	text-embedding-3-small, text-embedding-3-large, text-embedding-ada-002
Jina AI	jina-embeddings-v4, jina-embeddings-v3, jina-embeddings-v2-base-en
Hugging Face	BAAI/bge-small-en-v1.5, BAAI/bge-base-en-v1.5, thenlper/gte-small
Local	sentence-transformers/all-MiniLM-L6-v2, bert-base-uncased

References

Astral. (2024). *Uv: An extremely fast python package and project manager*. https://github.com/astral-sh/uv

Asudani, D. S., Nagwani, N. K., & Singh, P. (2023). Impact of word embedding models on text analytics in deep learning environment: A review. *Artificial intelligence review*, *56*(9), 10345–10425.

Auger, T., & Saroyan, E. (2024). Overview of the openai apis. In *Generative ai for web development: Building web applications powered by openai apis and next. js* (pp. 87–116). Springer.

Boateng, G. O., Neilands, T. B., Frongillo, E. A., Melgar-Quiñonez, H. R., & Young, S. L. (2018). Best practices for developing and validating scales for health, social, and behavioral research: A primer. *Frontiers in Public Health*, *6*, 149.

Carlini, N., Tramer, F., Wallace, E., Jagielski, M., Herbert-Voss, A., Lee, K., Roberts, A., Brown, T., Song, D., Erlingsson, U., et al. (2021). Extracting training data from large language models. *30th USENIX security symposium (USENIX Security 21)*, 2633–2650.

Chakrabarty, T., & Dhillon, P. S. (2026). Can good writing be generative? expert-level ai writing emerges through fine-tuning on high-quality books. https://arxiv.org/abs/2601.18353

Chen, B., Zhang, Z., Langréné, N., & Zhu, S. (2025). Unleashing the potential of prompt engineering in large language models: A comprehensive review. *Patterns*, *6*(6), 101260.

Christensen, A. P., Garrido, L. E., & Golino, H. (2023). Unique variable analysis: A network psychometrics method to detect local dependence. *Multivariate Behavioral Research*, *58*(6), 1165–1182. https://doi.org/10.1080/00273171.2023.2194606

Christensen, A. P., & Golino, H. (2021). Estimating the stability of psychological dimensions via bootstrap exploratory graph analysis: A monte carlo simulation and tutorial. *Psych*, *3*(3), 479–500. https://doi.org/10.3390/psych3030032

Clark, L. A., & Watson, D. (2016). Constructing validity: Basic issues in objective scale development. In A. E. Kazdin (Ed.), *Methodological issues and strategies in clinical research* (4th ed., pp. 187–203). American Psychological Association. https://doi.org/10.1037/14805-012

Danon, L., Diaz-Guilera, A., Duch, J., & Arenas, A. (2005). Comparing community structure identification. *Journal of statistical mechanics: Theory and experiment*, *2005*(09), P09008–P09008.

De Paoli, S. (2023). Improved prompting and process for writing user personas with llms, using qualitative interviews: Capturing behaviour and personality traits of users. *arXiv preprint arXiv:2310.06391*.

Evstafev, E. (2025). The paradox of stochasticity: Limited creativity and computational decoupling in temperature-varied llm outputs of structured fictional data. *arXiv preprint arXiv:2502.08515*.

Fenn, J., Tan, C.-S., & George, S. (2020). Development, validation and translation of psychological tests. *BJPsych advances*, *26*(5), 306–315.

Foygel, R., & Drton, M. (2010). Extended bayesian information criteria for gaussian graphical models. *Advances in neural information processing systems*, *23*.

Friedman, J., Hastie, T., & Tibshirani, R. (2008). Sparse inverse covariance estimation with the graphical lasso. *Biostatistics*, *9*(3), 432–441.

Garrido, L. E., Russell-Lasalandra, L., & Golino, H. (2025). Estimating dimensional structure in generative psychometrics: Comparing pca and network methods using large language model item embeddings. *PsyArXiv Preprints*.

Golino, H., & Christensen, A. (2025). *Eganet: Exploratory graph analysis – a framework for estimating the number of dimensions in multivariate data using network psychometrics* [R package version 2.1.1]. https://doi.org/10.32614/CRAN.package.EGAnet

Golino, H. F., & Epskamp, S. (2017). Exploratory graph analysis: A new approach for estimating the number of dimensions in psychological research. *PLoS ONE*, *12*(6), e0174035. https://doi.org/10.1371/journal.pone.0174035

Götz, F. M., Maertens, R., Loomba, S., & Van Der Linden, S. (2024). Let the algorithm speak: How to use neural networks for automatic item generation in psychological scale development. *Psychological Methods*, *29*(3), 494.

Groq. (n.d.). Lpu architecture [Accessed: 2026-03-29]. https://groq.com/lpu-architecture

Güven, G. Ö., Yilmaz, Ş., & Inceoğlu, F. (2024). Determining medical students' anxiety and readiness levels about artificial intelligence. *Heliyon*, *10*(4).

Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2019). The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*.

Hommel, B. E., Wollang, F.-J. M., Kotova, V., Zacher, H., & Schmukle, S. C. (2022). Transformer-based deep neural language modeling for construct-specific automatic item generation. *Psychometrika*, *87*(2), 749–772.

Hu, Z., Rostami, M., & Thomason, J. (2026). Expert personas improve llm alignment but damage accuracy: Bootstrapping intent-based persona routing with prism. *arXiv preprint arXiv:2603.18507*.

Jiang, H., Zhang, X., Cao, X., Breazeal, C., Roy, D., & Kabbara, J. (2024). Personallm: Investigating the ability of large language models to express personality traits. *Findings of the association for computational linguistics: NAACL 2024*, 3605–3627.

John, O. P., & Srivastava, S. (1999). The Big Five trait taxonomy: History, measurement, and theoretical perspectives. In L. A. Pervin & O. P. John (Eds.), *Handbook of personality: Theory and research* (2nd, pp. 102–138). Guilford Press.

Keane, D., & McNaughton, R. B. (2026). Using generative ai to enhance psychometric scale development in market research. *International Journal of Market Research*, *68*(2), 194–218.

Kong, A., Zhao, S., Chen, H., Li, Q., Qin, Y., Sun, R., Zhou, X., Wang, E., & Dong, X. (2024). Better zero-shot reasoning with role-play prompting. *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 4099–4113.

Li, J., & Huang, J.-S. (2020). Dimensions of artificial intelligence anxiety based on the integrated fear acquisition theory. *Technology in society*, *63*, 101410.

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023). Let's verify step by step. https://arxiv.org/abs/2305.20050

Liu, A., Diab, M., & Fried, D. (2024). Evaluating large language model biases in persona-steered generation. *Findings of the Association for Computational Linguistics: ACL 2024*, 9832–9850.

Liu, X., & Liu, Y. (2025). Developing and validating a scale of artificial intelligence anxiety among chinese efl teachers. *European Journal of Education*, *60*(1), e12902.

Martin Kowal, J., Hurley Bryant, K., Segall, D., & Kantrowitz, T. (2025). Harnessing generative ai for assessment item development: Comparing ai-generated and human-authored items. *International Journal of Selection and Assessment*, *33*(3), e70021.

Massara, G. P., Di Matteo, T., & Aste, T. (2016). Network filtering for big data: Triangulated maximally filtered graph. *Journal of complex Networks*, *5*(2), 161–178.

OpenAI. (2024a). Api reference: Chat completions. https://platform.openai.com/docs/api-reference/chat/create

OpenAI. (2024b). What are tokens and how to count them? https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them

Patel, D., Timsina, P., Raut, G., Freeman, R., levin, M. A., Nadkarni, G. N., Glicksberg, B. S., & Klang, E. (2024). Exploring temperature effects on large language models across various clinical tasks. *medRxiv*, 2024–07.

Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, *2*(11), 559–572.

Pedersen, T. L. (2025). *Patchwork: The composer of plots* [R package version 1.3.2]. https://doi.org/10.32614/CRAN.package.patchwork

Peeperkorn, M., Kouwenhoven, T., Brown, D., & Jordanous, A. (2024). Is temperature the creativity parameter of large language models? *arXiv preprint arXiv:2405.00492*.

Renze, M. (2024). The effect of sampling temperature on problem solving in large language models. *Findings of the association for computational linguistics: EMNLP 2024*, 7346–7356.

Russell-Lasalandra, L. L., Christensen, A. P., & Golino, H. (2024). Generative psychometrics via ai-genie: Automatic item generation and validation via network-integrated evaluation. *PsyArXiv Preprints*.

Russell-Lasalandra, L. L., & Golino, H. (2026). Prompt engineering for scale development in generative psychometrics. *arXiv preprint arXiv:2603.15909*.

Shin, D., Kwon, S. K., & Lee, Y. (2025). Examining the efficacy of generative artificial intelligence in item generation: Comparative analysis of human-developed and ai-generated reading tests. *Education and Information Technologies*, *30*(16), 23981–24007.

Tao, C., Shen, T., Gao, S., Zhang, J., Li, Z., Hua, K., Hu, W., Tao, Z., & Ma, S. (2025). Llms are also effective embedding models: An in-depth overview. https://arxiv.org/abs/2412.12591

Tengler, K., & Brandhofer, G. (2025). Exploring the difference and quality of ai-generated versus human-written texts. *Discover Education*, *4*(1), 113.

Ushey, K., Allaire, J., & Tang, Y. (2026). *Reticulate: Interface to 'python'* [R package version 1.45.0]. https://rstudio.github.io/reticulate/

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*.

Wang, Y.-Y., & Wang, Y.-S. (2022). Development and validation of an artificial intelligence anxiety scale: An initial application in predicting motivated learning behavior. *Interactive Learning Environments*, *30*(4), 619–634.

Wickham, H. (2016). *Ggplot2: Elegant graphics for data analysis*. Springer-Verlag New York. https://ggplot2.tidyverse.org

Zhang, B., Horvath, S., et al. (2005). A general framework for weighted gene co-expression network analysis. *Statistical applications in genetics and molecular biology*, *4*(1), 1128.

Zheng, M., Pei, J., Logeswaran, L., Lee, M., & Jurgens, D. (2024). When "a helpful assistant" is not really helpful: Personas in system prompts do not improve performances of large language models. *Findings of the Association for Computational Linguistics: EMNLP 2024*, 15126–15154.

Zhu, Y., Li, J., Li, G., Zhao, Y., Jin, Z., & Mei, H. (2024). Hot or cold? adaptive temperature sampling for code generation with large language models. *Proceedings of the AAAI Conference on Artificial Intelligence*, *38*(1), 437–445.